

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN



KHAI THÁC DỮ LIỆU VÀ ỨNG DỤNG

Đồ án 2

KHAI THÁC TẬP PHỔ BIẾN

Lớp: CQ2023/21

Giảng viên: Th.S Lê Nhựt Nam

Nhóm 5: Cao Tiến Thành - 23120088
Đỗ Quốc Thịnh - 23120089
Cao Thanh Bình - 23120216
Nguyễn Văn Chiến - 23120219
Đặng Nguyễn Thái Đạt - 23120227

Thành phố Hồ Chí Minh, 05/2026

LỜI CẢM ƠN

Chúng em xin gửi lời cảm ơn chân thành đến thầy Th.S Lê Nhật Nam đã trực tiếp giảng dạy và hướng dẫn tận tình cho các nhóm trong suốt quá trình thực hiện Đồ án 2 môn Khai thác dữ liệu và ứng dụng. Sự đồng hành và giúp đỡ kịp thời của thầy là nguồn lực to lớn để giúp chúng em hoàn thành đồ án này một cách trọn vẹn. Sự nhiệt tình và những lời chỉ bảo về việc sử dụng AI có trách nhiệm của thầy càng làm chúng em cảm thấy phải cố gắng nỗ lực nhiều hơn. Chúng em mong được tiếp nhận tất cả góp ý từ thầy để có một báo cáo hoàn chỉnh nhất có thể. Kính chúc thầy luôn luôn có thật nhiều sức khỏe!

Nhóm 5

Thành phố Hồ Chí Minh, tháng 05 năm 2026

Mục lục

Lời cảm ơn	i
Danh mục thuật ngữ	iii
Danh mục ký hiệu	v
Danh mục hình ảnh	vi
Danh mục bảng	vii
1 Nền tảng lý thuyết	1
1.1 Bài toán Khai thác tập phổ biến	1
1.2 Phân tích thuật toán LCM	2
2 Ví dụ minh họa chạy tay	15
2.1 Ví dụ Cơ sở	15
2.2 Minh họa một trường hợp đặc biệt của LCM	21
3 Cài đặt	25
3.1 Phiên bản A0	25
3.2 Phiên bản A1	28
3.3 Phiên bản A2	33
4 Thực nghiệm và đánh giá	38
4.1 Mục tiêu và phương pháp thực nghiệm	38
4.2 Tập dữ liệu benchmark	39
4.3 Thực nghiệm 1: Kiểm tra tính đúng đắn	39
4.4 Thực nghiệm 2: Thời gian chạy theo minsup	41
4.5 Thực nghiệm 3: Số lượng tập phổ biến đóng theo minsup	45
4.6 Thực nghiệm 4: Sử dụng bộ nhớ	46
4.7 Thực nghiệm 5: Khả năng mở rộng	47
4.8 Thực nghiệm 6: Ảnh hưởng của độ dài giao dịch trung bình	49
4.9 Tổng hợp phân tích và đề xuất tối ưu hóa	50
5 Ứng dụng – Phân tích giỏ hàng	53
5.1 Cấu hình thực nghiệm	53
5.2 Support, confidence và lift	54
5.3 Kết quả thực nghiệm	55
5.4 Phân tích các luật kết hợp nổi bật	55
5.5 Ý nghĩa kinh doanh	56
5.6 Nhận xét	58
Tài liệu tham khảo	60

DANH MỤC THUẬT NGỮ

Tiếng Anh	Tiếng Việt
Frequent Itemset Mining (FIM)	Khai thác tập phổ biến
Transaction Database	Cơ sở dữ liệu giao dịch
Frequent Itemset	Tập phổ biến
Minimum support threshold	Ngưỡng độ hỗ trợ tối thiểu
Closed Itemset	Tập đóng
Maximal Itemset	Tập tối đại
Support	Độ hỗ trợ
Confidence	Độ tin cậy
Absolute Support	Độ hỗ trợ tuyệt đối
Relative Support	Độ hỗ trợ tương đối
Anti-monotone property	Tính chất đơn điệu giảm
Linear time Closed itemset Miner	Thuật toán LCM
transaction	giao dịch
itemset	tập mục
i-prefix	tiền tố theo i
closure	phép đóng
Vertical Database	Cơ sở dữ liệu dạng dọc
Association rule	Luật kết hợp
TID-list	Danh sách id giao dịch
sorted merge	Hợp nhất theo thứ tự
BitVector	Vector bit
chunk-level	Mức khối
FCA	Phân tích khái niệm hình thức
Recall	Độ phủ
Precision	Độ chính xác
Exact match rate	Tỉ lệ khớp hoàn toàn
stress test	Kiểm thử căng thẳng
benchmark	Điểm chuẩn
empty set	Tập rỗng
ppc-extension	Mở rộng ppc
PPCE	Mảng phân phát theo P
occurrence deliver	phân phát theo danh sách giao dịch
database reduction	thu gọn cơ sở dữ liệu
hypercube decomposition	phân rã siêu khối
DFS	Tìm kiếm theo chiều sâu
adaptive representation	biểu diễn thích nghi
peak memory	Bộ nhớ đỉnh
scalability	Khả năng mở rộng
garbage collection (GC)	Thu gom rác
warm-up	Khởi động (làm nóng)
SIMD	Vector hóa song song
multithreading	Đa luồng
allocated bytes	Số byte cấp phát
density	Mật độ
synthetic dataset	Bộ dữ liệu tổng hợp

log-log plot	Đồ thị log-log
semilogy plot	Đồ thị bán log
Jaccard index	Chỉ số Jaccard
JIT compiler	Trình biên dịch JIT
log scale	Thang log
AVX-512	Tập lệnh AVX-512

DANH MỤC KÝ HIỆU

Ký hiệu	Ý nghĩa
$minsup$	ngưỡng hỗ trợ tối thiểu
$minconf$	ngưỡng tự tin tối thiểu
$\mathcal{D}$	cơ sở dữ liệu giao dịch
T_i	giao dịch thứ i
$\mathcal{T}(P)$	denotation của P
$freq(P)$	tần suất của P
$clo(P)$	phép đóng (closure) của P
$tail(P)$	item có chỉ số lớn nhất trong P
$P^{(i)}$	i-tiền tố của P
TID	Mã giao dịch
e	item đang được xem xét

DANH MỤC HÌNH ẢNH

1.1	Minh họa cây ppc-extension.	6
1.2	Minh họa Hypercube Decomposition	8
1.3	Vai trò của bucket trong occurrence deliver	9
1.4	Minh họa các cấu trúc dữ liệu chính của LCM với node hiện tại $P = \{1\}$	10
1.5	Dòng thời gian phát triển từ Apriori đến LCM v2	14
2.1	Cây duyệt DFS của LCM trên cơ sở dữ liệu siêu thị với $\theta = 3$	19
2.2	Cây duyệt LCM trong trường hợp dữ liệu phân mảnh thành hai cụm item đồng xuất hiện.	23
4.1	Thời gian chạy trên tập dữ liệu chess theo minsup.	41
4.2	Thời gian chạy trên tập dữ liệu mushrooms theo minsup.	42
4.3	Thời gian chạy trên tập dữ liệu retail theo minsup.	43
4.4	Thời gian chạy trên tập dữ liệu accidents theo minsup.	44
4.5	Số tập phổ biến đóng $ C $ theo minsup trên bốn tập dữ liệu.	45
4.6	Số byte cấp phát bởi ba phiên bản trên bốn tập dữ liệu tại minsup thấp nhất của mỗi tập.	47
4.7	Thời gian chạy theo kích thước dữ liệu trên retail (minsup = 1%).	48
4.8	Thời gian chạy theo độ dài giao dịch L trên cơ sở dữ liệu tổng hợp.	49

DANH MỤC BẢNG

1.1	Cơ sở dữ liệu giao dịch minh họa	3
1.2	Minh họa occurrence deliver tại $P = \{1\}$	7
1.3	Minh họa gộp giao dịch giống nhau khi database reduction	7
1.4	Minh họa các item trong $H(P)$ với $P = \{1\}$	8
1.5	Các cấu trúc dữ liệu chính trong LCM	9
1.6	Tóm tắt độ phức tạp thời gian của LCM	12
1.7	So sánh lịch sử LCM với các thuật toán trước đó	13
2.1	Ánh xạ mặt hàng sang chỉ số item	15
2.2	Cơ sở dữ liệu giao dịch siêu thị	15
2.3	Các bucket khởi tạo từ node gốc $P_0 = \emptyset$	16
2.4	Bảng trace các bước duyệt DFS của LCM	17
2.5	Danh sách tập phổ biến đóng được LCM khai thác	20
2.6	Liệt kê thủ công toàn bộ tập phổ biến với $\theta = 3$	20
2.7	Đối chiếu các tập phổ biến với phép đóng tương ứng	21
2.8	Cơ sở dữ liệu giao dịch cho trường hợp đặc biệt	22
4.1	Thông số môi trường thực nghiệm	38
4.2	Đặc điểm bốn tập dữ liệu benchmark	39
4.3	Lưới minsup cho từng tập dữ liệu	39
4.4	Tóm tắt kết quả đối chiếu Julia LCM và SPMF trên lưới chính	40
4.5	Kết quả kiểm thử căng thẳng với A0	40
4.6	Tăng tốc so với SPMF-Java tại minsup thấp nhất của mỗi tập	44
4.7	Bộ nhớ đỉnh (peak RSS) tại minsup trung vị	46
4.8	Đặc điểm bốn cơ sở dữ liệu tổng hợp	49
4.9	Điểm mạnh và điểm yếu của ba phiên bản Julia so với SPMF	50
5.1	Top – 10 association rules theo lift	55

CHƯƠNG 1

NỀN TẢNG LÝ THUYẾT

1.1 Bài toán Khai thác tập phổ biến

Trước hết, ta nhắc lại một số định nghĩa cơ bản sau:

Định nghĩa 1.1.1 (Cơ sở dữ liệu giao dịch). Gọi $\mathcal{I} = \{i_1, i_2, \dots, i_m\}$ là tập tất cả các item. Một giao dịch T là một tập con của $\mathcal{I}$ ($T \subseteq \mathcal{I}$). Một cơ sở dữ liệu giao dịch $\mathcal{D}$ là một tập hữu hạn các giao dịch $\mathcal{D} = \{T_1, T_2, \dots, T_n\}$.

Định nghĩa 1.1.2 (Độ hỗ trợ). Cho một itemset $X \subseteq \mathcal{I}$, độ hỗ trợ của X trong cơ sở dữ liệu $\mathcal{D}$ được xác định theo hai dạng:

- **Độ hỗ trợ tuyệt đối:** Là số lượng giao dịch trong $\mathcal{D}$ chứa X .

$$sup_{abs}(X) = |\{T_i \in \mathcal{D} \mid X \subseteq T_i\}| \quad (1.1)$$

- **Độ hỗ trợ tương đối:** Là tỉ lệ giao dịch trong $\mathcal{D}$ chứa X .

$$sup_{rel}(X) = \frac{|\{T_i \in \mathcal{D} \mid X \subseteq T_i\}|}{|\mathcal{D}|} \quad (1.2)$$

Định nghĩa 1.1.3 (Tập phổ biến). Với một ngưỡng hỗ trợ tối thiểu $minsup$ cho trước (dạng tuyệt đối hoặc tương đối), một itemset X được gọi là tập phổ biến nếu $sup(X) \geq minsup$.

Định nghĩa 1.1.4 (Tập đóng và Tập tối đại). Để giảm bớt sự bùng nổ số lượng tập phổ biến, ta định nghĩa các khái niệm sau:

- **Tập đóng:** Một itemset X là tập đóng nếu không tồn tại tập cha $Y \supset X$ sao cho $sup(Y) = sup(X)$. Nói cách khác, itemset X là đóng nếu không thể thêm item nào nữa vào X mà độ hỗ trợ của nó vẫn giữ nguyên.
- **Tập tối đại:** Một itemset X là tập tối đại nếu X là tập phổ biến và không tồn tại tập cha $Y \supset X$ mà Y cũng là tập phổ biến. Nói cách khác, một tập phổ biến P là tối đại nếu không thể thêm item nào nữa vào P mà nó vẫn phổ biến.

Quan hệ giữa các loại: $Frequent \supseteq Closed \supseteq Maximal$, xét trong tập hợp các kết quả phổ biến.

Định lý 1.1.1 (Tính chất Apriori). *Tính chất đơn điệu giảm của độ hỗ trợ phát biểu rằng: Nếu một itemset X là phổ biến, thì mọi tập con của nó cũng là phổ biến.*

Chứng minh. Giả sử $Y \subseteq X$. Với mọi transaction $T_i \in \mathcal{D}$, nếu $X \subseteq T_i$ thì hiển nhiên $Y \subseteq T_i$, theo tính chất bắc cầu của quan hệ bao hàm. Do đó, số lượng transaction chứa Y luôn lớn hơn hoặc bằng số lượng transaction chứa X :

$$sup(Y) \geq sup(X) \quad (1.3)$$

Vì X là tập phổ biến nên $sup(X) \geq minsup$, suy ra $sup(Y) \geq minsup$. Vậy Y cũng là tập phổ biến. $\square$

1.2 Phân tích thuật toán LCM

Thuật toán **LCM** (*Linear time Closed itemset Miner*) được Uno và cộng sự đề xuất vào năm 2004 nhằm khai thác hiệu quả các **tập phổ biến đóng**. Điểm khác biệt quan trọng của LCM là thuật toán không sinh toàn bộ tập phổ biến rồi mới lọc ra tập đóng, mà xây dựng trực tiếp một cây tìm kiếm trên không gian các tập đóng. Cơ chế sinh cây này dựa trên kỹ thuật *prefix preserving closure extension* (ppc-extension), nhờ đó mỗi tập phổ biến đóng được sinh đúng một lần và không cần lưu toàn bộ các kết quả đã tìm được để kiểm tra trùng lặp [3].

1.2.1 Ký hiệu và định nghĩa nền tảng

Gọi $\mathcal{I} = \{1, 2, \dots, n\}$ là tập item được đánh chỉ số và $\mathcal{T} = \{t_1, t_2, \dots, t_m\}$ là cơ sở dữ liệu giao dịch, trong đó mỗi giao dịch $t_i \subseteq \mathcal{I}$. Kích thước cơ sở dữ liệu giao dịch là tổng độ dài tất cả các giao dịch:

$$\|\mathcal{T}\| = \sum_{t \in \mathcal{T}} |t|.$$

Định nghĩa 1.2.1 (Denotation và tần suất). Với itemset $P \subseteq \mathcal{I}$, **denotation** của P là tập các giao dịch chứa P :

$$\mathcal{T}(P) = \{t \in \mathcal{T} \mid P \subseteq t\}.$$

Tần suất của P là:

$$\text{frq}(P) = |\mathcal{T}(P)|.$$

Với ngưỡng độ hỗ trợ tối thiểu θ , P là tập phổ biến nếu $\text{frq}(P) \geq \theta$.

Hai tính chất cơ bản thường được dùng trong LCM là:

$$\mathcal{T}(P \cup Q) = \mathcal{T}(P) \cap \mathcal{T}(Q),$$

$$P \subseteq Q \Rightarrow \mathcal{T}(Q) \subseteq \mathcal{T}(P).$$

Nói cách khác, itemset càng lớn thì số giao dịch chứa nó càng không thể tăng.

Định nghĩa 1.2.2 (Closure). Closure của itemset P , ký hiệu $\text{clo}(P)$, là giao của tất cả giao dịch chứa P :

$$\text{clo}(P) = \bigcap_{t \in \mathcal{T}(P)} t.$$

Itemset P là **tập đóng** nếu và chỉ nếu:

$$\text{clo}(P) = P.$$

Phép đóng có các tính chất quan trọng sau:

1. Nếu $P \subseteq Q$ thì $\text{clo}(P) \subseteq \text{clo}(Q)$.
2. Nếu $\mathcal{T}(P) = \mathcal{T}(Q)$ thì $\text{clo}(P) = \text{clo}(Q)$.
3. $\text{clo}(\text{clo}(P)) = \text{clo}(P)$.

4. $\text{clo}(P)$ là tập đóng nhỏ nhất chứa P .
5. P là tập đóng khi và chỉ khi $\text{clo}(P) = P$.

Định nghĩa 1.2.3 (Tail, i -tiền tố và closure extension). Với itemset P :

- $\text{tail}(P)$ là item có chỉ số lớn nhất trong P .
- $P^{(i)} = P \cap \{1, 2, \dots, i\}$ là i -tiền tố của P , tức tập con của P gồm các item có chỉ số không vượt quá i .
- Q là closure extension của P nếu tồn tại $e \notin P$ sao cho $Q = \text{clo}(P \cup \{e\})$.

1.2.2 Ý tưởng cốt lõi

Trong khai thác tập phổ biến, số lượng tập phổ biến có thể tăng rất nhanh khi ngưỡng hỗ trợ tối thiểu nhỏ. Tuy nhiên, nhiều tập phổ biến mang thông tin dư thừa. Ví dụ, nếu hai itemset X và Y có cùng tập giao dịch chứa chúng, tức $\mathcal{T}(X) = \mathcal{T}(Y)$, thì chúng có cùng độ hỗ trợ và biểu diễn cùng một nhóm giao dịch. Khi đó, tập lớn nhất trong nhóm tương đương này chính là **tập đóng**.

Ý tưởng của LCM có thể tóm tắt qua ba bước:

1. Với một itemset hiện tại P , thêm một item e để tạo ứng viên $P \cup \{e\}$.
2. Tính phép đóng $P' = \text{clo}(P \cup \{e\})$ để nhảy trực tiếp đến một tập đóng.
3. Kiểm tra điều kiện ppc-extension để quyết định P' có phải là con hợp lệ của P trong cây tìm kiếm hay không.

Nhờ cách làm này, LCM không đi trên cây của mọi tập phổ biến, mà đi trên cây của các tập phổ biến đóng. Đây là lý do thuật toán có thể đạt hiệu quả cao, đặc biệt trên các bộ dữ liệu thưa, nơi số lượng item lớn nhưng mỗi giao dịch chỉ chứa ít item.

Ví dụ 1.2.1 (Trực giác về phép đóng). Xét cơ sở dữ liệu giao dịch trong Bảng 1.1.

Bảng 1.1: Cơ sở dữ liệu giao dịch minh họa

TID	Items
t_1	$\{1, 2, 3\}$
t_2	$\{1, 2, 3, 4\}$
t_3	$\{2, 3, 5\}$
t_4	$\{1, 3\}$

Với $P = \{1, 2\}$, ta có $\mathcal{T}(P) = \{t_1, t_2\}$. Do đó:

$$\text{clo}(\{1, 2\}) = t_1 \cap t_2 = \{1, 2, 3\} \cap \{1, 2, 3, 4\} = \{1, 2, 3\}.$$

Điều này có nghĩa là trong mọi giao dịch chứa $\{1, 2\}$ đều luôn xuất hiện thêm item 3. Vì vậy, $\{1, 2\}$ chưa đóng, còn $\{1, 2, 3\}$ là tập đóng.

1.2.3 Kỹ thuật ppc-extension

Closure tail

Định nghĩa 1.2.4 (Closure tail). Với tập đóng P , **closure tail** của P , ký hiệu $\text{clo_tail}(P)$, là item $i \in P$ có chỉ số nhỏ nhất sao cho:

$$\text{clo}(P^{(i)}) = P.$$

Trực giác: $\text{clo_tail}(P)$ là vị trí sớm nhất trong P mà chỉ cần lấy tiền tố đến đó, phép đóng đã tái tạo được toàn bộ P .

Ví dụ 1.2.2 (Tính closure tail). Xét lại Bảng 1.1 và tập đóng $P = \{1, 2, 3\}$. Ta có:

$$\mathcal{T}(P) = \{t_1, t_2\}, \quad \text{clo}(P) = \{1, 2, 3\} = P.$$

Ta lần lượt kiểm tra các tiền tố:

- Với $i = 1$: $P^{(1)} = \{1\}$, $\mathcal{T}(\{1\}) = \{t_1, t_2, t_4\}$, nên

$$\text{clo}(\{1\}) = t_1 \cap t_2 \cap t_4 = \{1, 3\} \neq P.$$

- Với $i = 2$: $P^{(2)} = \{1, 2\}$, $\mathcal{T}(\{1, 2\}) = \{t_1, t_2\}$, nên

$$\text{clo}(\{1, 2\}) = t_1 \cap t_2 = \{1, 2, 3\} = P.$$

Vì $i = 2$ là chỉ số nhỏ nhất thỏa điều kiện, ta có:

$$\text{clo_tail}(\{1, 2, 3\}) = 2.$$

Định nghĩa ppc-extension

Định nghĩa 1.2.5 (Prefix preserving closure extension). Cho P là một tập đóng. Tập đóng P' là **ppc-extension** của P theo item e nếu thỏa mãn đồng thời:

1. $e \notin P$ và $e > \text{clo_tail}(P)$;
2. $P' = \text{clo}(P \cup \{e\})$;
3. $P'^{(e-1)} = P^{(e-1)}$.

Điều kiện thứ ba là phần quan trọng nhất. Nó yêu cầu phép đóng sau khi thêm e không được làm thay đổi phần tiền tố trước e . Nếu P' xuất hiện thêm một item nhỏ hơn e mà P chưa có, thì P' nên được sinh từ một nhánh khác có tiền tố nhỏ hơn, nên LCM bỏ qua nhánh hiện tại để tránh trùng lặp.

Ví dụ 1.2.3 (Minh họa ppc-extension). Xét lại cơ sở dữ liệu giao dịch trong Bảng 1.1. Giả sử ngưỡng hỗ trợ tối thiểu là $\theta = 2$. Ta xét itemset $P = \{1, 3\}$.

Trước hết, ta kiểm tra tính đóng của P . Các giao dịch chứa P là:

$$\mathcal{T}(\{1, 3\}) = \{t_1, t_2, t_4\}.$$

Do đó:

$$\text{clo}(\{1, 3\}) = t_1 \cap t_2 \cap t_4 = \{1, 2, 3\} \cap \{1, 2, 3, 4\} \cap \{1, 3\} = \{1, 3\}.$$

Vì $\text{clo}(P) = P$, suy ra $P = \{1, 3\}$ là một tập đóng.

Tiếp theo, ta tính $\text{clo_tail}(P)$. Theo định nghĩa:

$$P^{(i)} = P \cap \{1, 2, \dots, i\}.$$

Với $i = 1$, ta có:

$$P^{(1)} = \{1, 3\} \cap \{1\} = \{1\}.$$

Các giao dịch chứa item 1 là:

$$\mathcal{T}(\{1\}) = \{t_1, t_2, t_4\}.$$

Suy ra:

$$\text{clo}(\{1\}) = t_1 \cap t_2 \cap t_4 = \{1, 3\} = P.$$

Vì $i = 1$ là chỉ số nhỏ nhất thỏa mãn $\text{clo}(P^{(i)}) = P$, nên:

$$\text{clo_tail}(P) = 1.$$

Bây giờ, ta xét mở rộng P bằng item $e = 2$. Vì:

$$e = 2 > \text{clo_tail}(P) = 1,$$

điều kiện thứ nhất của ppc-extension được thỏa mãn.

Ta tính closure của $P \cup \{2\}$:

$$P \cup \{2\} = \{1, 2, 3\}.$$

Các giao dịch chứa $\{1, 2, 3\}$ là:

$$\mathcal{T}(\{1, 2, 3\}) = \{t_1, t_2\}.$$

Do đó:

$$\text{clo}(P \cup \{2\}) = \text{clo}(\{1, 2, 3\}) = t_1 \cap t_2 = \{1, 2, 3\}.$$

Đặt:

$$P' = \{1, 2, 3\}.$$

Ta có:

$$P' = \text{clo}(P \cup \{2\}).$$

Cuối cùng, ta kiểm tra điều kiện bảo toàn tiền tố:

$$P'^{(e-1)} = P^{(e-1)}.$$

Vì $e = 2$, nên $e - 1 = 1$. Khi đó:

$$P'^{(1)} = \{1, 2, 3\} \cap \{1\} = \{1\},$$

và:

$$P^{(1)} = \{1, 3\} \cap \{1\} = \{1\}.$$

Suy ra:

$$P'^{(1)} = P^{(1)} = \{1\}.$$

Vậy cả ba điều kiện đều được thỏa mãn:

$$\begin{cases} 2 > \text{clo_tail}(P), \\ P' = \text{clo}(P \cup \{2\}), \\ P'^{(2-1)} = P^{(2-1)}. \end{cases}$$

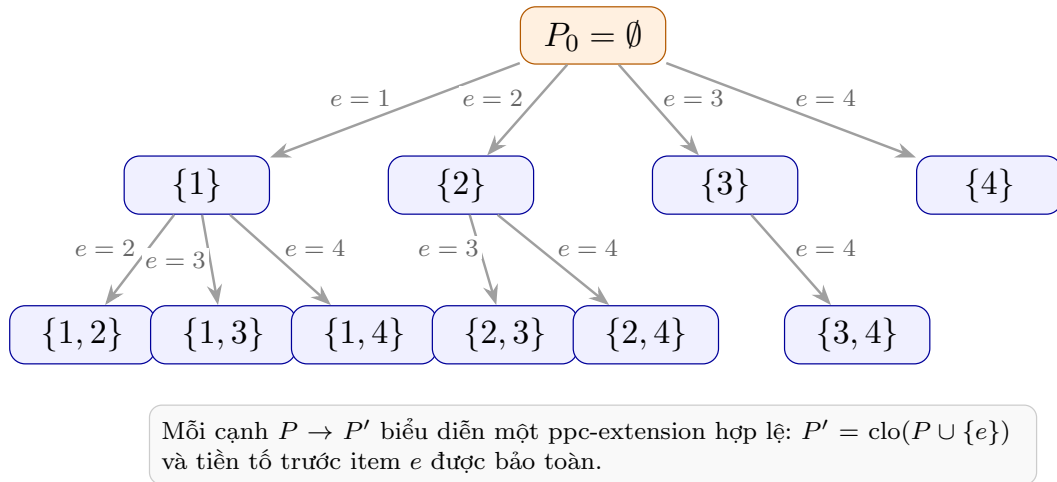
Do đó:

$\{1, 2, 3\}$ là một ppc-extension của $\{1, 3\}$ theo item $e = 2$.

Ý nghĩa của ví dụ trên là: từ tập đóng cha $P = \{1, 3\}$, LCM thêm item 2, sau đó lấy closure để thu được tập đóng con $P' = \{1, 2, 3\}$. Điều kiện bảo toàn tiền tố đảm bảo rằng P' được sinh ra đúng từ nhánh của P , tránh việc cùng một tập đóng bị sinh lặp lại từ nhiều nhánh khác nhau.

Cây ppc-extension

Gọi $P_0 = \text{clo}(\emptyset)$ là tập đóng gốc. Bài báo LCM chứng minh rằng mọi tập đóng $P' \neq P_0$ đều là ppc-extension của đúng một tập đóng cha P . Hơn nữa, $\text{frq}(P) > \text{frq}(P')$, nên quan hệ cha-con này tạo thành một cây có gốc trên toàn bộ không gian tập đóng phổ biến [3].



Hình 1.1: Minh họa cây ppc-extension.

Mỗi node là một tập đóng, mỗi cạnh biểu diễn một ppc-extension hợp lệ từ tập đóng cha sang tập đóng con.

Hệ quả trực tiếp là LCM có thể duyệt DFS trên cây này mà không cần lưu danh sách các tập đóng đã sinh. Điều này giúp giảm mạnh bộ nhớ so với các phương pháp lưu kết quả trung gian để kiểm tra trùng lặp.

1.2.4 Các kỹ thuật tối ưu hóa

Bên cạnh ppc-extension, LCM v2 còn dùng nhiều kỹ thuật thực thi để giảm thời gian chạy trong thực tế. Ba kỹ thuật quan trọng nhất là **occurrence deliver**, **anytime**

database reduction và hypercube decomposition.**Occurrence deliver**

Tại một node P , thuật toán cần tính $\text{frq}(P \cup \{e\})$ cho nhiều item e . Nếu đếm riêng từng e , thuật toán phải quét cơ sở dữ liệu nhiều lần. LCM dùng **occurrence deliver** để tính tất cả các denotation $\mathcal{T}(P \cup \{e\})$ trong một lần duyệt $\mathcal{T}(P)$.

Ý tưởng là tạo một bucket cho mỗi item mở rộng e . Với mỗi giao dịch $t \in \mathcal{T}(P)$, nếu t chứa item e thì đưa t vào $\text{Bucket}[e]$. Sau khi duyệt xong, ta có:

$$\text{Bucket}[e] = \mathcal{T}(P \cup \{e\}).$$

Bảng 1.2: Minh họa occurrence deliver tại $P = \{1\}$

Giao dịch đang xét	Bucket[2]	Bucket[3]	Bucket[4]
Khởi tạo	$\emptyset$	$\emptyset$	$\emptyset$
$t_1 = \{1, 2, 3\}$	$\{t_1\}$	$\{t_1\}$	$\emptyset$
$t_2 = \{1, 2, 4\}$	$\{t_1, t_2\}$	$\{t_1\}$	$\{t_2\}$
$t_3 = \{1, 3, 4\}$	$\{t_1, t_2\}$	$\{t_1, t_3\}$	$\{t_2, t_3\}$

Với cách này, LCM tính đồng thời nhiều tần suất trong một lần duyệt dữ liệu. Đây là một trong các lý do LCM nhanh trên dữ liệu thưa.

Anytime database reduction

Ở lời gọi đệ quy ứng với itemset P , mọi node con đều chứa P . Vì vậy, LCM có thể thu gọn cơ sở dữ liệu trước khi đi sâu hơn:

1. Xóa các giao dịch không chứa P .
2. Xóa các item không còn đủ ngưỡng hỗ trợ tối thiểu trong ngữ cảnh hiện tại.
3. Gộp các giao dịch giống nhau và lưu thêm trọng số.

Kỹ thuật này được gọi là **anytime database reduction** vì việc thu gọn được thực hiện liên tục tại các node trong cây tìm kiếm, không chỉ ở bước tiền xử lý ban đầu. Trong LCM v2, kỹ thuật này được bổ sung để cải thiện hiệu năng trên các bộ dữ liệu dày, vốn là điểm yếu của phiên bản LCM đầu tiên [3].

Bảng 1.3: Minh họa gộp giao dịch giống nhau khi database reduction

Trước khi gộp		Sau khi gộp	
TID	Items	Items	Trọng số
t_1	$\{2, 3, 4\}$	$\{2, 3, 4\}$	2
t_2	$\{2, 3, 4\}$	$\{2, 4\}$	1
t_3	$\{2, 4\}$		

Hypercube decomposition

Hypercube decomposition được dùng chủ yếu trong biến thể LCMfreq để khai thác toàn bộ tập phổ biến. Với tập phổ biến P , định nghĩa:

$$H(P) = \{e \mid e > \text{tail}(P), \mathcal{T}(P) = \mathcal{T}(P \cup \{e\})\}.$$

Nếu $e \in H(P)$, việc thêm e vào P không làm thay đổi denotation. Do đó, với mọi $Q \subseteq H(P)$, ta có:

$$\mathcal{T}(P \cup Q) = \mathcal{T}(P).$$

Vì vậy, thuật toán có thể xuất cùng lúc toàn bộ các itemset nằm giữa P và $P \cup H(P)$ thay vì sinh từng tập riêng lẻ.

Ví dụ 1.2.4 (Minh họa Hypercube Decomposition). Xét lại cơ sở dữ liệu giao dịch trong Bảng 1.1. Giả sử ngưỡng hỗ trợ tối thiểu là $\theta = 2$.

Ta chọn tập phổ biến hiện tại:

$$P = \{1\}.$$

Các giao dịch chứa P là:

$$\mathcal{T}(\{1\}) = \{t_1, t_2, t_4\}.$$

Do đó:

$$\text{freq}(\{1\}) = 3 \geq \theta.$$

Theo định nghĩa trong kỹ thuật Hypercube Decomposition, ta xét tập:

$$H(P) = \{e \mid e > \text{tail}(P) \text{ và } \mathcal{T}(P) = \mathcal{T}(P \cup \{e\})\}.$$

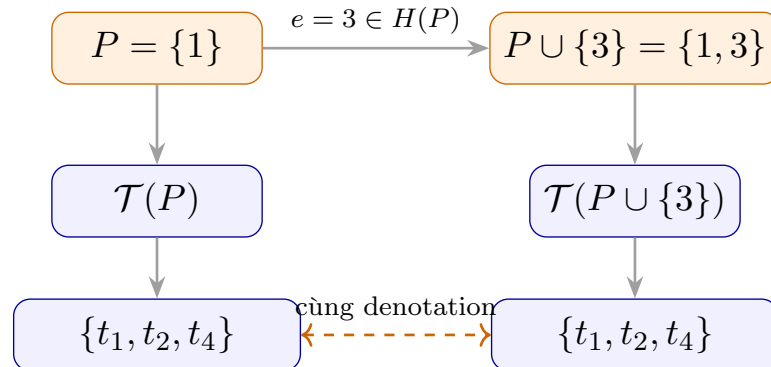
Vì $P = \{1\}$ nên:

$$\text{tail}(P) = 1.$$

Ta lần lượt xét các item $e > 1$. Item $e \in H(P)$ nếu $\mathcal{T}(P) = \mathcal{T}(P \cup \{e\}) = \{t_1, t_2, t_4\}$. Kết quả được tóm tắt trong bảng dưới:

Bảng 1.4: Minh họa các item trong $H(P)$ với $P = \{1\}$

Item e	$P \cup \{e\}$	$\mathcal{T}(P \cup \{e\})$	Thuộc $H(P)$?
2	$\{1, 2\}$	$\{t_1, t_2\}$	Không
3	$\{1, 3\}$	$\{t_1, t_2, t_4\}$	Có
4	$\{1, 4\}$	$\{t_2\}$	Không
5	$\{1, 5\}$	$\emptyset$	Không



Hình 1.2: Minh họa Hypercube Decomposition

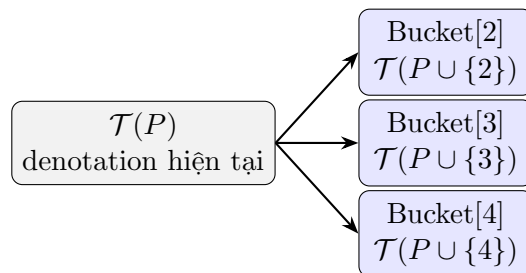
Như vậy, thay vì phải xử lý riêng từng itemset, LCM có thể gom các itemset có cùng denotation thành một khối trong không gian tìm kiếm. Trong ví dụ trên, hai itemset $\{1\}$ và $\{1, 3\}$ tạo thành một hypercube nhỏ. Việc xuất đồng thời các tập phổ biến trong hypercube giúp giảm số lần đếm tần suất và giảm số nhánh cần duyệt trong cây tìm kiếm.

1.2.5 Cấu trúc dữ liệu chính

Thuật toán LCM không dùng cấu trúc phức tạp như FP-tree, mà chủ yếu dùng **mảng**. Lý do là thao tác chính của LCM không phải tìm kiếm giao dịch bất kỳ, mà là duyệt các giao dịch trong denotation của itemset hiện tại.

Bảng 1.5: Các cấu trúc dữ liệu chính trong LCM

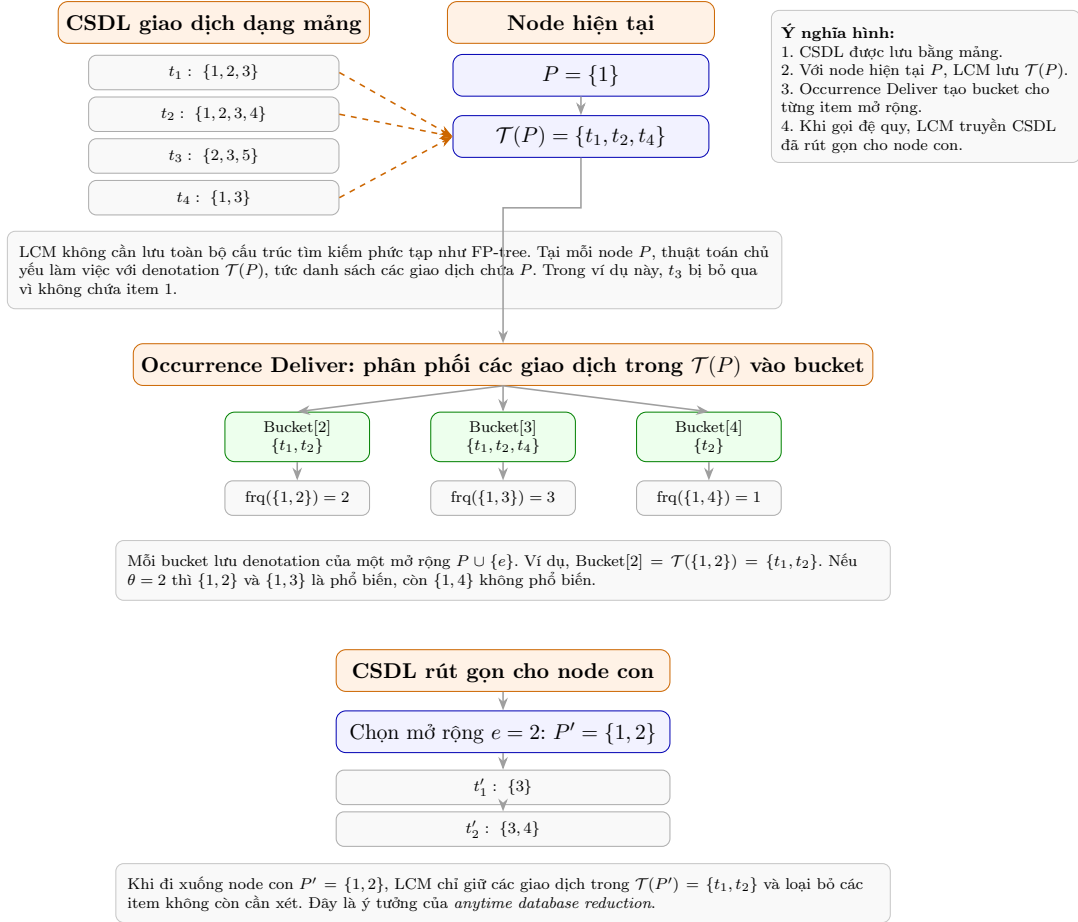
Cấu trúc	Vai trò	Minh họa / ghi chú
Mảng giao dịch	Lưu cơ sở dữ liệu giao dịch sau khi sắp xếp và thu gọn.	Mỗi giao dịch là một danh sách item; thao tác chính là duyệt tuần tự.
Denotation $\mathcal{T}(P)$	Lưu danh sách giao dịch chứa itemset hiện tại P .	Đây là dữ liệu đầu vào của occurrence deliver tại node P .
Bucket Bucket[e]	Lưu $\mathcal{T}(P \cup \{e\})$ cho từng item mở rộng e .	Sau một lần duyệt $\mathcal{T}(P)$, kích thước bucket chính là tần suất của extension tương ứng.
CSDL thu gọn	Lưu phần dữ liệu còn cần thiết cho cây con hiện tại.	Gồm các giao dịch còn lại, các item còn phổ biến và trọng số nếu có giao dịch bị gộp.
Interior intersection	Hỗ trợ tính closure khi nhiều giao dịch giống nhau được gộp.	Khi gộp giao dịch, cần giữ thông tin giao của các giao dịch gốc để tính closure chính xác.



Occurrence deliver phân phát mỗi giao dịch vào các bucket tương ứng với item mà nó chứa.

Hình 1.3: Vai trò của bucket trong occurrence deliver

Xét lại ví dụ ở Bảng 1.1

Hình 1.4: Minh họa các cấu trúc dữ liệu chính của LCM với node hiện tại $P = \{1\}$.

Việc dùng mảng giúp LCM có chi phí thao tác thấp và khởi tạo nhanh. Bài báo nhấn mạnh rằng LCM không cần tìm kiếm transaction trong cây như FP-tree; thay vào đó, thuật toán chủ yếu duyệt tuần tự các giao dịch liên quan đến itemset hiện tại [3].

1.2.6 Mã giả thuật toán LCM

LCM bắt đầu từ $P_0 = \text{clo}(\emptyset)$. Tại mỗi node, thuật toán xuất P , tính các extension bằng occurrence deliver, đóng từng extension, rồi kiểm tra điều kiện ppc-extension trước khi đệ quy.

Algorithm 1 LCM($P, \mathcal{T}(P), \theta$)**Require:** Tập đóng phổ biến hiện tại P , denotation $\mathcal{T}(P)$, ngưỡng hỗ trợ θ **Ensure:** Xuất tất cả tập phổ biến đóng trong cây con gốc P

```

1: Xuất  $P$ 
2:  $\mathcal{T}' \leftarrow \text{DATABASEREDUCTION}(\mathcal{T}(P), P, \theta)$ 
3: Tính Bucket[ $e$ ] cho mọi item mở rộng  $e$  bằng OCCURRENCEDELIVER( $\mathcal{T}', P$ )
4: for mỗi item  $e \notin P$  thỏa  $e > \text{clo\_tail}(P)$  do
5:   if |Bucket[ $e$ ]  $\geq \theta$  then
6:      $P' \leftarrow \text{clo}(P \cup \{e\})$ 
7:     if  $P^{(e-1)} = P^{(e-1)}$  then
8:       LCM( $P', \text{Bucket}[e], \theta$ )
9:     end if
10:  end if
11: end for

```

Giải thích mã giả:

- Dòng 1 xuất P vì mọi node trong cây ppc-extension đều là một tập phổ biến đóng hợp lệ.
- Dòng 2 thu gọn cơ sở dữ liệu để giảm chi phí cho các bước sau.
- Dòng 3 dùng occurrence deliver để tính đồng thời denotation của nhiều extension.
- Dòng 5 kiểm tra điều kiện phổ biến của extension.
- Dòng 6 tính closure để nhảy trực tiếp từ $P \cup \{e\}$ đến tập đóng P' .
- Dòng 7 kiểm tra điều kiện bảo toàn tiền tố. Đây là bước quyết định để tránh sinh trùng lặp.

Lưu ý rằng khi $P' = \text{clo}(P \cup \{e\})$, ta có $\mathcal{T}(P') = \mathcal{T}(P \cup \{e\})$. Vì vậy, denotation truyền xuống lời gọi đệ quy có thể lấy trực tiếp từ Bucket[e].

1.2.7 Phân tích độ phức tạp**Độ phức tạp thời gian**

Gọi $\mathcal{C}$ là tập tất cả các tập phổ biến đóng được sinh bởi LCM. Nhờ ppc-extension, mỗi phần tử trong $\mathcal{C}$ tương ứng với đúng một node trong cây tìm kiếm. Vì vậy, số lần xử lý node là $|\mathcal{C}|$.

Tại một node P , chi phí chính đến từ occurrence deliver và tính closure. Với occurrence deliver, chi phí có thể viết là:

$$O \left(\sum_{t \in \mathcal{T}(P)} |t \cap \{e \mid e > \text{tail}(P)\}| \right).$$

Trong trường hợp xấu, chi phí tại một node bị chặn bởi $O(\|\mathcal{T}\|)$. Do đó:

$$T_{\text{worst}} = O(|\mathcal{C}| \cdot \|\mathcal{T}\|).$$

Tuy nhiên, trong thực tế, LCM thường nhanh hơn nhiều nhờ anytime database reduction. Khi xét kích thước cơ sở dữ liệu thu gọn tại từng node P , có thể viết gần đúng hơn:

$$T_{\text{practical}} = O\left(\sum_{P \in \mathcal{C}} \|\mathcal{T}'_P\|\right),$$

trong đó $\mathcal{T}'_P$ là cơ sở dữ liệu đã được thu gọn tại node P .

Bảng 1.6: Tóm tắt độ phức tạp thời gian của LCM

Trường hợp	Phân tích
Tốt nhất	<i>minsup</i> cao, ít tập đóng, dữ liệu được thu gọn mạnh. Cây tìm kiếm nhỏ và denotation ở các node giảm nhanh.
Trung bình	Chi phí phụ thuộc vào tổng kích thước các cơ sở dữ liệu thu gọn trên toàn bộ cây: $O\left(\sum_{P \in \mathcal{C}} \ \mathcal{T}'_P\ \right)$.
Xấu nhất	Nếu dữ liệu khó thu gọn và số tập đóng lớn, chi phí bị chặn bởi $O(\mathcal{C} \cdot \ \mathcal{T}\)$.

Điểm quan trọng không phải là LCM luôn tuyến tính theo kích thước dữ liệu, mà là thời gian chạy được kiểm soát theo số lượng tập đóng đầu ra. Bài báo gọi đây là ưu điểm thời gian tuyến tính theo số lượng tập phổ biến đóng [3], trong khi nhiều thuật toán trước có thể phát sinh chi phí siêu tuyến tính do sinh nhiều tập không đóng hoặc kiểm tra trùng lặp bằng bộ nhớ kết quả. Tham khảo các thuật toán [1] [2] [4]

Độ phức tạp không gian

LCM duyệt cây theo DFS nên không cần lưu toàn bộ tập kết quả. Không gian chính gồm:

1. đường đi đệ quy hiện tại;
2. cơ sở dữ liệu thu gọn tại các tầng đang xét;
3. các bucket phục vụ occurrence deliver;
4. các mảng phụ trợ để tính closure và kiểm tra ppc-extension.

Vì không cần lưu tất cả tập đóng đã sinh, bộ nhớ của LCM không phụ thuộc trực tiếp vào $|\mathcal{C}|$. Đây là ưu điểm lớn khi số lượng tập phổ biến đóng rất lớn.

Các yếu tố ảnh hưởng đến hiệu năng

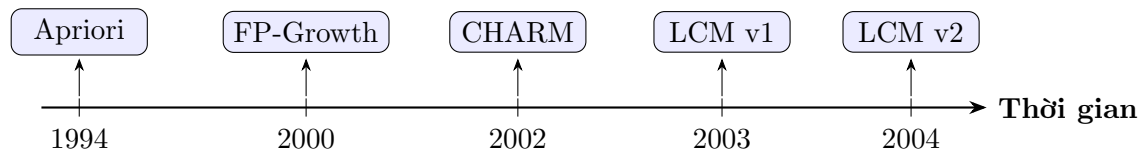
1. **Ngưỡng *minsup*:** *minsup* càng thấp thì số lượng tập phổ biến đóng càng lớn, làm cây tìm kiếm rộng và sâu hơn.
2. **Độ thưa/dày của dữ liệu:** Dữ liệu thưa thường phù hợp với LCM vì occurrence deliver và mảng hoạt động hiệu quả. Dữ liệu quá dày có thể có lợi hơn cho FP-tree hoặc bitmap.
3. **Chiều dài giao dịch trung bình:** Giao dịch càng dài thì mỗi lần occurrence deliver càng tốn nhiều thao tác phân phát.
4. **Khả năng thu gọn dữ liệu:** Nếu database reduction loại bỏ được nhiều item/giao dịch, các node sâu sẽ xử lý dữ liệu nhỏ hơn rất nhiều.

1.2.8 So sánh lịch sử

LCM nằm trong quá trình phát triển của các thuật toán khai thác tập phổ biến. Bảng 1.7 tóm tắt vai trò của một số thuật toán tiêu biểu và điểm LCM cải thiện.

Bảng 1.7: So sánh lịch sử LCM với các thuật toán trước đó

Thuật toán	Ý tưởng chính	Hạn chế	LCM cải thiện
Apriori	Sinh ứng viên theo từng mức và dùng tính chất Apriori để tĩa.	Cần nhiều lần quét CSDL; số ứng viên có thể rất lớn.	LCM dùng DFS và sinh trực tiếp tập đóng bằng closure.
FP-Growth	Nén dữ liệu bằng FP-tree và khai thác theo pattern growth.	FP-tree có chi phí xây dựng và không phải lúc nào cũng phù hợp với dữ liệu thưa.	LCM dùng mảng đơn giản, khởi tạo nhanh và hiệu quả trên dữ liệu thưa.
CHARM	Khai thác tập đóng bằng tidset/diffset và phép giao TID.	Có thể cần lưu nhiều cấu trúc trung gian để kiểm tra quan hệ giữa các tập.	LCM dùng ppc-extension để sinh mỗi tập đóng đúng một lần mà không cần lưu toàn bộ kết quả.
LCM v1	Dùng ppc-extension để liệt kê các tập phổ biến đóng.	Chưa có database reduction nên yếu hơn trên dữ liệu dày.	LCM v2 bổ sung any-time database reduction, cải thiện closure operation và các kỹ thuật cho LCMmax.



Hình 1.5: Dòng thời gian phát triển từ Apriori đến LCM v2

Hạn chế còn lại của LCM

Mặc dù LCM có nhiều ưu điểm, thuật toán vẫn có một số hạn chế:

- Với dữ liệu rất dày và *minsup* thấp, số lượng tập đóng có thể rất lớn, khiến thời gian chạy vẫn tăng mạnh.
- LCM dùng mảng nên không tận dụng được khả năng nén prefix của FP-tree trong những bộ dữ liệu có nhiều giao dịch chia sẻ tiền tố dài.
- Các biến thể khai thác tập tối đại như LCMmax vẫn phụ thuộc vào hiệu quả của bước kiểm tra maximality, vốn được chính tác giả xem là hướng có thể tiếp tục cải thiện.

Tóm lại, LCM giải quyết một điểm nghẽn quan trọng của các thuật toán trước: sinh trùng lặp và sinh quá nhiều tập không đóng. Bằng ppc-extension, thuật toán xây dựng một cây duy nhất trên các tập phổ biến đóng; bằng occurrence deliver và anytime database reduction, thuật toán giảm mạnh chi phí đếm hỗ trợ tại từng node. Đây là lý do LCM trở thành một trong những thuật toán tiêu biểu cho bài toán khai thác tập phổ biến đóng.

CHƯƠNG 2

VÍ DỤ MINH HỌA CHẠY TAY

2.1 Ví dụ Cơ sở

Mục tiêu của ví dụ này là minh họa cách LCM liệt kê các tập phổ biến đóng bằng các ý tưởng đã trình bày ở Chương 1: **closure**, **ppc-extension**, **occurrence deliver** và duyệt cây theo chiều sâu DFS.

2.1.1 Cơ sở dữ liệu minh họa

Xét cơ sở dữ liệu giao dịch siêu thị gồm 5 mặt hàng. Để thuận tiện cho thuật toán, các mặt hàng được ánh xạ sang chỉ số nguyên tăng dần như trong Bảng 2.1.

Bảng 2.1: Ánh xạ mặt hàng sang chỉ số item

Ký hiệu	Sản phẩm	Chỉ số item
A	Bánh mì	1
B	Sữa	2
C	Trứng	3
D	Bơ	4
E	Cà phê	5

Cơ sở dữ liệu giao dịch được cho trong Bảng 2.2. Trong ví dụ này, ta chọn ngưỡng hỗ trợ tối thiểu: $\theta = 3$.

Một itemset được xem là phổ biến nếu nó xuất hiện trong ít nhất 3 giao dịch.

Bảng 2.2: Cơ sở dữ liệu giao dịch siêu thị

Giao dịch	Sản phẩm	Dạng chỉ số
t_1	A, B, C	$\{1, 2, 3\}$
t_2	A, C, D	$\{1, 3, 4\}$
t_3	B, C, E	$\{2, 3, 5\}$
t_4	A, B, C, D	$\{1, 2, 3, 4\}$
t_5	A, B, D	$\{1, 2, 4\}$
t_6	A, B, C, E	$\{1, 2, 3, 5\}$

2.1.2 Quy tắc chạy tay theo LCM

Tại mỗi node ứng với một tập đóng hiện tại P , LCM xét các item mở rộng e thỏa $e > \text{clo_tail}(P)$. Với mỗi item e , thuật toán thực hiện các bước sau:

1. Tạo tập tạm thời $X = P \cup \{e\}$.
2. Tính denotation $\mathcal{T}(X)$ bằng kỹ thuật *occurrence deliver* hoặc giao các TID-set đã có.
3. Nếu $|\mathcal{T}(X)| < \theta$, loại nhánh vì không phổ biến.
4. Nếu đủ phổ biến, tính tập đóng mới:

$$P' = \text{clo}(X).$$

5. Kiểm tra điều kiện ppc-extension:

$$P'^{(e-1)} = P^{(e-1)}.$$

Nếu điều kiện này đúng, P' là con hợp lệ của P trong cây ppc-extension; ngược lại, nhánh bị loại vì tập đóng đó sẽ được sinh ở một nhánh khác.

2.1.3 Bước 1: Khởi tạo từ tập đóng gốc

LCM bắt đầu từ tập đóng gốc:

$$P_0 = \text{clo}(\emptyset).$$

Khi đó $\mathcal{T}(P_0)$ chính là toàn bộ cơ sở dữ liệu:

$$\mathcal{T}(P_0) = \{t_1, t_2, t_3, t_4, t_5, t_6\}.$$

Tiến hành giao tất cả các giao dịch từ t_1 đến t_6 để tính P_0 :

$$P_0 = \emptyset.$$

Từ P_0 , ta dùng occurrence deliver để tính nhanh bucket cho từng item mở rộng. Kết quả được trình bày trong Bảng 2.3.

Bảng 2.3: Các bucket khởi tạo từ node gốc $P_0 = \emptyset$

Item e	Denotation	$\text{freq}(e)$	Kết luận
1	$\{t_1, t_2, t_4, t_5, t_6\}$	5	Nhận
2	$\{t_1, t_3, t_4, t_5, t_6\}$	5	Nhận
3	$\{t_1, t_2, t_3, t_4, t_6\}$	5	Nhận
4	$\{t_2, t_4, t_5\}$	3	Nhận
5	$\{t_3, t_6\}$	2	Loại

Như vậy, từ node gốc chỉ có các item 1, 2, 3, 4 được xét tiếp. Item 5 bị loại vì support nhỏ hơn θ .

2.1.4 Bước 2: Duyệt DFS theo ppc-extension

Bảng 2.4 tóm tắt toàn bộ quá trình chạy tay của LCM. Mỗi dòng tương ứng với một lần thử mở rộng P bằng item e .

Bảng 2.4: Bảng trace các bước duyệt DFS của LCM

Bước	Cha P	e	$X = P \cup \{e\}$	$\mathcal{T}(X)$	sup	$P' = \text{clo}(X)$	$P'^{(e-1)} = P^{(e-1)}$?	Kết luận
Khởi tạo	$\emptyset$	1	$\{1\}$	$\{t_1, t_2, t_4, t_5, t_6\}$	5	$\{1\}$	True	Nhận
1	$\{1\}$	2	$\{1, 2\}$	$\{t_1, t_4, t_5, t_6\}$	4	$\{1, 2\}$	True	Nhận
2	$\{1, 2\}$	3	$\{1, 2, 3\}$	$\{t_1, t_4, t_6\}$	3	$\{1, 2, 3\}$	True	Nhận
3	$\{1, 2, 3\}$	4	$\{1, 2, 3, 4\}$	$\{t_4\}$	1	–	–	Loại
4	$\{1, 2\}$	4	$\{1, 2, 4\}$	$\{t_4, t_5\}$	2	–	–	Loại
5	$\{1\}$	3	$\{1, 3\}$	$\{t_1, t_2, t_4, t_6\}$	4	$\{1, 3\}$	True	Nhận
6	$\{1, 3\}$	4	$\{1, 3, 4\}$	$\{t_2, t_4\}$	2	–	–	Loại
7	$\{1\}$	4	$\{1, 4\}$	$\{t_2, t_4, t_5\}$	3	$\{1, 4\}$	True	Nhận
8	$\emptyset$	2	$\{2\}$	$\{t_1, t_3, t_4, t_5, t_6\}$	5	$\{2\}$	True	Nhận
9	$\{2\}$	3	$\{2, 3\}$	$\{t_1, t_3, t_4, t_6\}$	4	$\{2, 3\}$	True	Nhận
10	$\{2, 3\}$	4	$\{2, 3, 4\}$	$\{t_4\}$	1	–	–	Loại
11	$\{2\}$	4	$\{2, 4\}$	$\{t_4, t_5\}$	2	–	–	Loại
12	$\emptyset$	3	$\{3\}$	$\{t_1, t_2, t_3, t_4, t_6\}$	5	$\{3\}$	True	Nhận
13	$\{3\}$	4	$\{3, 4\}$	$\{t_2, t_4\}$	2	–	–	Loại
14	$\emptyset$	4	$\{4\}$	$\{t_2, t_4, t_5\}$	3	$\{1, 4\}$	False	Loại

2.1.5 Giải thích chi tiết một số bước quan trọng

Nhánh hợp lệ: từ $\emptyset$ sang $\{1\}$

Từ node gốc $P = \emptyset$, xét item $e = 1$:

$$X = P \cup \{1\} = \{1\}.$$

Các giao dịch chứa $\{1\}$ là:

$$\mathcal{T}(\{1\}) = \{t_1, t_2, t_4, t_5, t_6\}, \quad |\mathcal{T}(\{1\})| = 5 \geq 3.$$

Lấy giao của các giao dịch này:

$$\begin{aligned} \text{clo}(\{1\}) &= t_1 \cap t_2 \cap t_4 \cap t_5 \cap t_6 \\ &= \{1\}. \end{aligned}$$

Đặt $P' = \{1\}$. Vì $e = 1$ nên ta kiểm tra tiền tố trước e :

$$P'^{(0)} = \emptyset = P^{(0)}.$$

Do đó $\{1\}$ là ppc-extension hợp lệ của $\emptyset$ và được ghi nhận là một tập phổ biến đóng.

Nhánh sâu: từ $\{1\}$ sang $\{1, 2\}$ rồi sang $\{1, 2, 3\}$

Sau khi nhận $P = \{1\}$, LCM duyệt sâu theo DFS. Xét $e = 2$:

$$X = \{1\} \cup \{2\} = \{1, 2\}.$$

Ta có:

$$\mathcal{T}(\{1, 2\}) = \{t_1, t_4, t_5, t_6\}, \quad \sup(\{1, 2\}) = 4.$$

Closure của X là:

$$\text{clo}(\{1, 2\}) = t_1 \cap t_4 \cap t_5 \cap t_6 = \{1, 2\}.$$

Kiểm tra ppc-extension với $e = 2$:

$$\{1, 2\}^{(1)} = \{1\} = \{1\}^{(1)}.$$

Vậy $\{1, 2\}$ được nhận.

Từ $P = \{1, 2\}$, xét tiếp $e = 3$:

$$\mathcal{T}(\{1, 2, 3\}) = \{t_1, t_4, t_6\}, \quad \sup = 3.$$

Do đó:

$$\text{clo}(\{1, 2, 3\}) = t_1 \cap t_4 \cap t_6 = \{1, 2, 3\}.$$

Kiểm tra tiền tố:

$$\{1, 2, 3\}^{(2)} = \{1, 2\} = \{1, 2\}^{(2)}.$$

Vậy $\{1, 2, 3\}$ cũng là một ppc-extension hợp lệ.

Tương tự cho các bước 5, 7, 8, 9, 12.

Nhánh bị loại do không đủ support

Ở bước 3, từ $P = \{1, 2, 3\}$, xét tiếp $e = 4$:

$$X = \{1, 2, 3, 4\}$$

Các giao dịch chứa X là:

$$\mathcal{T}(X) = \{t_4\}.$$

Do $\sup(X) = 1 < \theta = 3$ nên LCM loại nhánh này ngay mà không cần tính closure.

Tương tự, ta backtrack về $\{1, 2\}$, từ $P = \{1, 2\}$, xét $e = 4$:

$$X = \{1, 2, 4\}.$$

Các giao dịch chứa X là:

$$\mathcal{T}(X) = \{t_4, t_5\}.$$

Do $\sup(X) = 2 < \theta = 3$ nên LCM loại nhánh này ngay mà không cần tính closure.

Tương tự cho các nhánh tại các bước 6, 10, 11, 13

Nhánh bị loại do vi phạm điều kiện ppc-extension

Từ node gốc $P = \emptyset$, xét item $e = 4$:

$$X = \{4\}, \quad \mathcal{T}(\{4\}) = \{t_2, t_4, t_5\}.$$

Vì support bằng 3, tập này đủ phổ biến. Tuy nhiên, closure của nó là:

$$\begin{aligned} \text{clo}(\{4\}) &= t_2 \cap t_4 \cap t_5 \\ &= \{1, 4\}. \end{aligned}$$

Đặt $P' = \{1, 4\}$. Với $e = 4$, điều kiện ppc-extension yêu cầu:

$$P'^{(3)} = P^{(3)}.$$

Nhưng:

$$P'^{(3)} = \{1\}, \quad P^{(3)} = \emptyset.$$

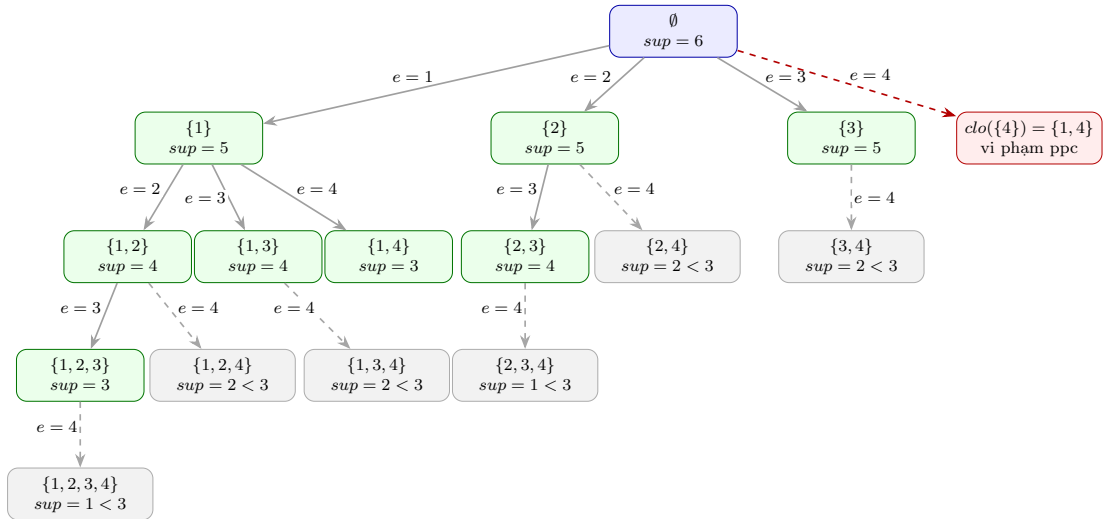
Do đó:

$$P'^{(3)} \neq P^{(3)}.$$

Nhánh này bị loại vì vi phạm điều kiện bảo toàn tiền tố. Về trực giác, closure của $\{4\}$ đã kéo thêm item 1 nhỏ hơn 4 vào tập kết quả. Điều này cho thấy tập đóng $\{1, 4\}$ không nên được sinh từ nhánh gốc theo item 4, mà phải được sinh từ nhánh bắt đầu bằng item 1. Thật vậy, $\{1, 4\}$ đã được nhận ở nhánh $\{1\} \rightarrow \{1, 4\}$.

2.1.6 Hình minh họa cây duyệt DFS của LCM

Hình 2.1 minh họa cây duyệt DFS tương ứng với quá trình chạy tay ở trên. Các node màu xanh là các tập phổ biến đóng được nhận, node màu đỏ là nhánh bị loại do vi phạm ppc-extension, và node màu xám là nhánh bị loại vì không đủ support.



Hình 2.1: Cây duyệt DFS của LCM trên cơ sở dữ liệu siêu thị với $\theta = 3$.

2.1.7 Kết quả cuối cùng

Sau khi duyệt DFS theo cây ppc-extension, các tập phổ biến đóng thu được được tổng hợp trong Bảng 2.5. Tập rỗng $\emptyset$ là tập đóng gốc nhưng thường không được đưa vào bảng kết quả khai thác mẫu.

Bảng 2.5: Danh sách tập phổ biến đóng được LCM khai thác

STT	Dạng chỉ số	Dạng sản phẩm	Support
1	{1}	{Bánh mì}	5
2	{2}	{Sữa}	5
3	{3}	{Trứng}	5
4	{1, 2}	{Bánh mì, Sữa}	4
5	{1, 3}	{Bánh mì, Trứng}	4
6	{1, 4}	{Bánh mì, Bơ}	3
7	{2, 3}	{Sữa, Trứng}	4
8	{1, 2, 3}	{Bánh mì, Sữa, Trứng}	3

Nhận xét. Qua ví dụ trên, có thể thấy LCM không sinh tất cả tập phổ biến rồi mới lọc tập đóng. Thay vào đó, thuật toán sinh trực tiếp các tập đóng thông qua ppc-extension. Điều kiện bảo toàn tiền tố giúp tránh sinh trùng lặp, chẳng hạn nhánh $\emptyset \rightarrow \{4\}$ bị loại vì closure của $\{4\}$ là $\{1, 4\}$, vốn đã thuộc nhánh bắt đầu từ item 1. Ngoài ra, các nhánh không đủ support như $\{1, 2, 3, 4\}$, $\{1, 2, 4\}$, $\{1, 3, 4\}$, $\{2, 3, 4\}$,... được cắt bỏ sớm, giúp giảm số phép tính closure và số lời gọi đệ quy.

2.1.8 Kiểm tra chéo

Để kiểm tra chéo kết quả do thuật toán LCM sinh ra, ta liệt kê thủ công toàn bộ các tập phổ biến trên cơ sở dữ liệu trong Bảng 2.2 với ngưỡng hỗ trợ tối thiểu $\theta = 3$. Mục tiêu của bước này là xác nhận rằng các tập phổ biến đóng do LCM tìm được chính là các tập đóng nằm trong tập tất cả các tập phổ biến. Trước hết, ta tính support của từng item đơn lẻ như sau:

$$\text{sup}(\{1\}) = 5, \quad \text{sup}(\{2\}) = 5, \quad \text{sup}(\{3\}) = 5, \quad \text{sup}(\{4\}) = 3, \quad \text{sup}(\{5\}) = 2.$$

Vì $\text{sup}(\{5\}) = 2 < \theta$, item $\{5\}$ không phổ biến. Do đó, các tập chứa item 5 sẽ không được mở rộng thêm trong quá trình liệt kê thủ công các tập phổ biến.

Bảng 2.6: Liệt kê thủ công toàn bộ tập phổ biến với $\theta = 3$

Kích thước	Tập X	$\mathcal{T}(X)$	$\text{sup}(X)$
0	$\emptyset$	$\{t_1, t_2, t_3, t_4, t_5, t_6\}$	6
1	{1}	$\{t_1, t_2, t_4, t_5, t_6\}$	5
1	{2}	$\{t_1, t_3, t_4, t_5, t_6\}$	5
1	{3}	$\{t_1, t_2, t_3, t_4, t_6\}$	5
1	{4}	$\{t_2, t_4, t_5\}$	3
2	{1, 2}	$\{t_1, t_4, t_5, t_6\}$	4
2	{1, 3}	$\{t_1, t_2, t_4, t_6\}$	4
2	{1, 4}	$\{t_2, t_4, t_5\}$	3
2	{2, 3}	$\{t_1, t_3, t_4, t_6\}$	4
3	{1, 2, 3}	$\{t_1, t_4, t_6\}$	3

Từ Bảng 2.6, tập tất cả tập phổ biến là:

$$\mathcal{F}_{\theta=3} = \{\emptyset, \{1\}, \{2\}, \{3\}, \{4\}, \{1, 2\}, \{1, 3\}, \{1, 4\}, \{2, 3\}, \{1, 2, 3\}\}.$$

Tiếp theo, ta kiểm tra tính đóng của từng tập phổ biến bằng cách tính phép đóng. Một itemset X là tập đóng nếu và chỉ nếu $\text{clo}(X) = X$.

Bảng 2.7: Đối chiếu các tập phổ biến với phép đóng tương ứng

Tập phổ biến X	$\text{sup}(X)$	$\text{clo}(X)$	Kết luận
$\emptyset$	6	$\emptyset$	Đóng
$\{1\}$	5	$\{1\}$	Đóng
$\{2\}$	5	$\{2\}$	Đóng
$\{3\}$	5	$\{3\}$	Đóng
$\{4\}$	3	$\{1, 4\}$	Không đóng
$\{1, 2\}$	4	$\{1, 2\}$	Đóng
$\{1, 3\}$	4	$\{1, 3\}$	Đóng
$\{1, 4\}$	3	$\{1, 4\}$	Đóng
$\{2, 3\}$	4	$\{2, 3\}$	Đóng
$\{1, 2, 3\}$	3	$\{1, 2, 3\}$	Đóng

Từ Bảng 2.7, tập các tập phổ biến đóng là:

$$\mathcal{C}_{\theta=3} = \{\emptyset, \{1\}, \{2\}, \{3\}, \{1, 2\}, \{1, 3\}, \{1, 4\}, \{2, 3\}, \{1, 2, 3\}\}.$$

Điểm khác biệt duy nhất giữa tập tất cả tập phổ biến $\mathcal{F}_{\theta=3}$ và tập các tập phổ biến đóng $\mathcal{C}_{\theta=3}$ là ở tập $\{4\}$. Cụ thể:

$$\mathcal{T}(\{4\}) = \{t_2, t_4, t_5\}.$$

Khi đó:

$$\text{clo}(\{4\}) = t_2 \cap t_4 \cap t_5 = \{1, 3, 4\} \cap \{1, 2, 3, 4\} \cap \{1, 2, 4\} = \{1, 4\} \neq \{4\}.$$

Do đó, $\{4\}$ không phải là tập đóng vì tồn tại tập cha $\{1, 4\}$ mà:

$$\text{sup}(\{4\}) = \text{sup}(\{1, 4\}) = 3.$$

Kết quả kiểm tra chéo cho thấy các tập mà LCM sinh ra trong Bảng ?? khớp với toàn bộ các tập phổ biến đóng trong $\mathcal{C}_{\theta=3}$. Như vậy, quá trình duyệt DFS theo ppc-extension không bỏ sót tập phổ biến đóng nào, đồng thời loại bỏ đúng các tập phổ biến nhưng không đóng như tập $\{4\}$.

2.2 Minh họa một trường hợp đặc biệt của LCM

Trong phần này, ta xét một trường hợp đặc biệt nhằm làm rõ cách LCM xử lý các itemset có quan hệ xuất hiện đồng thời tuyệt đối. Đây là tình huống trong đó một số item luôn đi kèm nhau trong cùng một nhóm giao dịch, làm cho phép closure có thể mở rộng trực tiếp một itemset ban đầu thành một tập đóng lớn hơn. Trường hợp này đặc biệt quan trọng vì nó thể hiện rõ hai cơ chế cốt lõi của LCM: sinh tập đóng bằng phép closure và loại bỏ trùng lặp bằng điều kiện ppc-extension.

2.2.1 Cơ sở dữ liệu minh họa

Xét cơ sở dữ liệu giao dịch gồm bốn item với $minsup = 2$

$$\mathcal{I} = \{1, 2, 3, 4\},$$

Bảng 2.8: Cơ sở dữ liệu giao dịch cho trường hợp đặc biệt

TID	Items
t_1	$\{1, 2\}$
t_2	$\{1, 2\}$
t_3	$\{3, 4\}$
t_4	$\{3, 4\}$

Cơ sở dữ liệu trên có cấu trúc phân mảnh rõ rệt: hai item 1 và 2 luôn xuất hiện cùng nhau trong nhóm giao dịch $\{t_1, t_2\}$, trong khi hai item 3 và 4 luôn xuất hiện cùng nhau trong nhóm giao dịch $\{t_3, t_4\}$. Không có giao dịch nào chứa đồng thời item thuộc hai nhóm $\{1, 2\}$ và $\{3, 4\}$.

2.2.2 Xử lý của thuật toán LCM

Trước hết, LCM tính tập đóng gốc: $P_0 = \text{clo}(\emptyset)$

Vì không có item nào xuất hiện trong toàn bộ bốn giao dịch, nên $P_0 = \emptyset$

Đồng thời, support của từng item đơn lẻ là:

$$\text{sup}(1) = 2, \quad \text{sup}(2) = 2, \quad \text{sup}(3) = 2, \quad \text{sup}(4) = 2.$$

Do đó, cả bốn item đều thỏa mãn ngưỡng hỗ trợ tối thiểu $\theta = 2$.

Nhánh mở rộng với item $e = 1$

Từ tập đóng gốc $P_0 = \emptyset$, xét mở rộng: $X = P_0 \cup \{1\} = \{1\}$

Các giao dịch chứa item 1 là $\mathcal{T}(\{1\}) = \{t_1, t_2\}$

Vì cả hai giao dịch t_1 và t_2 đều chứa đồng thời item 2, nên $\text{clo}(\{1\}) = t_1 \cap t_2 = \{1, 2\}$

Đặt $P' = \{1, 2\}$, với $e = 1$, điều kiện bảo toàn tiền tố là $P'^{(e-1)} = P_0^{(e-1)}$

Do $e - 1 = 0$, ta có $P'^{(0)} = \emptyset$, $P_0^{(0)} = \emptyset$

Suy ra điều kiện ppc-extension được thỏa mãn. Vì vậy, LCM chấp nhận $\{1, 2\}$ là một tập phổ biến đóng.

Sau đó, thuật toán tiếp tục thử mở rộng sâu hơn từ $\{1, 2\}$ với các item lớn hơn. Tuy nhiên, các item 3 và 4 không xuất hiện cùng nhóm giao dịch với $\{1, 2\}$, nên mọi mở rộng như $\{1, 2, 3\}$ hoặc $\{1, 2, 4\}$ đều có support bằng 0. Do đó, nhánh này dừng lại.

Nhánh mở rộng với item $e = 2$

Tiếp theo, từ $P_0 = \emptyset$, xét $X = P_0 \cup \{2\} = \{2\}$

Ta có $\mathcal{T}(\{2\}) = \{t_1, t_2\}$

Vì item 1 cũng xuất hiện trong cả t_1 và t_2 , nên $\text{clo}(\{2\}) = \{1, 2\}$

Đặt $P' = \{1, 2\}$, với $e = 2$, điều kiện bảo toàn tiền tố là: $P'^{(1)} = P_0^{(1)}$

Ta có: $P'^{(1)} = \{1\}$, $P_0^{(1)} = \emptyset$

Do đó điều kiện ppc-extension bị vi phạm, LCM loại nhánh $e = 2$.

Điều này có ý nghĩa quan trọng: tập đóng $\{1, 2\}$ đã được sinh hợp lệ từ nhánh $e = 1$, nên khi xét nhánh $e = 2$, LCM phát hiện rằng closure sinh ra có chứa item nhỏ hơn e , làm thay đổi tiền tố. Nhờ đó, thuật toán tránh sinh trùng tập đóng $\{1, 2\}$.

Nhánh mở rộng với item $e = 3$ và $e = 4$

Tương tự, với $e = 3$, ta có: $X = \{3\}$, $\mathcal{T}(\{3\}) = \{t_3, t_4\}$

Vì item 4 xuất hiện đồng thời trong cả t_3 và t_4 , nên: $\text{clo}(\{3\}) = \{3, 4\}$

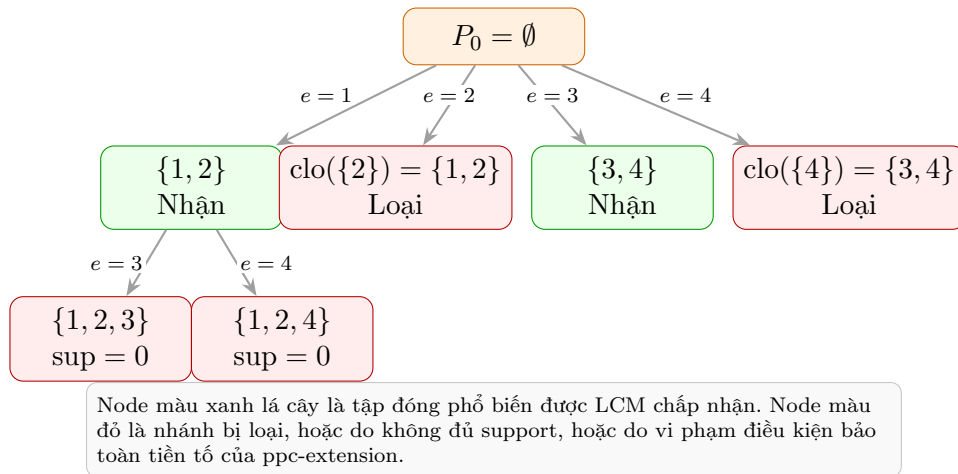
Điều kiện ppc-extension được thỏa mãn, do đó LCM chấp nhận $\{3, 4\}$ là một tập phổ biến đóng.

Ngược lại, với $e = 4$, ta có $\text{clo}(\{4\}) = \{3, 4\}$

Nhưng khi kiểm tra tiền tố với $e = 4$: $P'^{(3)} = \{3\}$, $P_0^{(3)} = \emptyset$

Do hai tiền tố khác nhau, nhánh $e = 4$ bị loại vì vi phạm điều kiện ppc-extension.

2.2.3 Minh họa cây duyệt của LCM



Hình 2.2: Cây duyệt LCM trong trường hợp dữ liệu phân mảnh thành hai cụm item đồng xuất hiện.

2.2.4 Kết quả và ý nghĩa của trường hợp đặc biệt

Kết quả cuối cùng của thuật toán là hai tập phổ biến đóng:

$$\{1, 2\}, \quad \{3, 4\}.$$

Đây là một trường hợp đặc biệt quan trọng vì ba lý do chính.

Thứ nhất, ví dụ cho thấy hiệu quả của phép bao đóng trong LCM. Khi xét item 1, thuật toán không dừng ở tập trung gian $\{1\}$ mà lấy closure để thu được trực tiếp

$\{1, 2\}$. Tương tự, từ item 3, thuật toán thu được ngay $\{3, 4\}$. Nhờ vậy, LCM không cần liệt kê toàn bộ các tập phổ biến trung gian trước khi kiểm tra tính đóng.

Thứ hai, ví dụ thể hiện rõ vai trò của điều kiện ppc-extension trong việc loại bỏ trùng lặp. Các nhánh $e = 2$ và $e = 4$ đều sinh ra những tập đóng đã có thể được sinh từ nhánh nhỏ hơn, lần lượt là $\{1, 2\}$ và $\{3, 4\}$. Điều kiện bảo toàn tiền tố phát hiện sự thay đổi tiền tố và loại bỏ các nhánh này mà không cần lưu danh sách các tập đóng đã sinh.

Thứ ba, trong cơ sở dữ liệu này, các tập đóng phổ biến tìm được cũng đồng thời là các tập phổ biến tối đại. Cụ thể, không tồn tại tập phổ biến nào chứa $\{1, 2\}$ hoặc $\{3, 4\}$ như một tập con thực sự. Do đó, ví dụ này minh họa một tình huống trong đó tập đóng phổ biến và tập phổ biến tối đại trùng nhau:

$$\mathcal{FCI} = \mathcal{MFI} = \{\{1, 2\}, \{3, 4\}\}.$$

Tuy nhiên, đây không phải là tính chất luôn đúng trong mọi cơ sở dữ liệu. Trong trường hợp tổng quát, một tập phổ biến đóng có thể chưa phải là tập phổ biến tối đại. Chính vì vậy, tình huống này được xem là một trường hợp đặc biệt giúp làm nổi bật khả năng rút gọn không gian tìm kiếm của LCM.

CHƯƠNG 3

CÀI ĐẶT

3.1 Phiên bản A0

Phiên bản A0 là phiên bản cài đặt cơ bản của thuật toán Linear time Closed itemset Miner (LCM) dựa trên cấu trúc cơ sở dữ liệu giao dịch dạng ngang.

Phiên bản này bám sát mô tả thuật toán lý thuyết trong bài báo gốc của tác giả Takeaki Uno.

3.1.1 Ý tưởng cài đặt

Ý tưởng cốt lõi của phiên bản này là triển khai một mô hình duyệt cây không gian tập đóng phổ biến (PPC-tree) không cần lưu vết các tập đóng đã tìm thấy.

Thuật toán bắt đầu từ bao đóng của tập rỗng và thực thi quá trình đệ quy sâu. Tại mỗi nút đệ quy ứng với tập đóng hiện tại P và cơ sở dữ liệu chiếu $T(P)$, thuật toán:

1. Sinh kết quả khai phá tương ứng với tập đóng hiện tại.
2. Áp dụng kỹ thuật Occurrence Deliver để phân phát các giao dịch chứa P vào các bucket tương ứng với mỗi phần tử mở rộng e lớn hơn $tail(P)$.
3. Đối với mỗi bucket có số giao dịch thỏa ngưỡng hỗ trợ tối thiểu, tiến hành tính bao đóng bằng cách giao toàn bộ các giao dịch trong bucket đó.
4. Thực hiện PPC check để xác định xem phép mở rộng có bảo toàn tiền tố hay không. Nếu hợp lệ, tiếp tục đi xuống đệ quy.

3.1.2 Tiền xử lý dữ liệu

Dữ liệu thô từ file SPMF được loại bỏ các dòng chú thích và chuyển thành danh sách giao dịch chuẩn hóa.

Phiên bản A0 áp dụng một bước tiền xử lý đặc trưng là đổi tên phần tử. Thuật toán đếm tần suất toàn cục của mọi item, loại bỏ các item không phổ biến và sắp xếp các item phổ biến theo tần suất xuất hiện tăng dần. Các item này được ánh xạ sang một hệ thống định danh số nguyên liên tục bắt đầu từ 1 đến num_freq_items (gọi là dense ID).

Cơ sở dữ liệu gốc sau đó được lọc và chuyển hoàn toàn sang các dense ID. Việc đổi tên này giúp các item ít phổ biến nhất có ID nhỏ nhất, từ đó tối ưu hóa việc phân chia bucket và cắt tỉa sớm các nhánh tìm kiếm.

3.1.3 Biểu diễn cơ sở dữ liệu bằng danh sách giao dịch ngang

Cơ sở dữ liệu trong phiên bản A0 được biểu diễn dưới dạng ngang truyền thống:

$$\mathcal{D} = \{t_1, t_2, \dots, t_m\}$$

trong đó mỗi giao dịch t_k là một mảng `VectorInt32` chứa các dense ID đã được sắp xếp tăng dần.

Khi thuật toán thực hiện đệ quy sâu đến nút đại diện cho tập đóng P , cơ sở dữ liệu chiều $T(P)$ được lưu giữ trực tiếp dưới dạng một vector chứa các giao dịch (các con trỏ hoặc bản sao mảng giao dịch của $\mathcal{D}$ có chứa P):

$$T(P) = \{t_k \in \mathcal{D} \mid P \subseteq t_k\}$$

Cấu trúc ngang này giúp thuật toán dễ dàng lập và phân phát trực tiếp các phần tử trong từng giao dịch ở bước Occurrence Deliver.

3.1.4 Tính độ hỗ trợ và phép đóng

Độ hỗ trợ của tập đóng hiện tại P được xác định trực tiếp bằng số lượng giao dịch trong cơ sở dữ liệu chiều $T(P)$:

$$\text{support}(P) = |T(P)|.$$

Trong quá trình Occurrence Deliver, đối với mỗi item mở rộng $e > \text{tail}(P)$, tập hợp các giao dịch chứa $P \cup \{e\}$ được thu thập trực tiếp trong `buckets[e]` (ký hiệu là $T(P \cup \{e\})$). Độ hỗ trợ của tập mở rộng tương lai được tính bằng:

$$\text{support}(P \cup \{e\}) = |T(P \cup \{e\})|.$$

Nếu $|T(P \cup \{e\})| \geq \text{minsup}$, phép tính bao đóng được thực thi. Bao đóng của tập $P \cup \{e\}$ được tính bằng cách lấy giao của tất cả các giao dịch chứa trong bucket đó:

$$\text{clo}(P \cup \{e\}) = \bigcap_{t \in T(P \cup \{e\})} t.$$

Kết quả của phép giao mảng này là một vector số nguyên đại diện cho tập đóng mới C .

3.1.5 Tính giao mảng giao dịch

Do các giao dịch được lưu trữ dưới dạng danh sách ngang các dense ID đã sắp xếp tăng dần, phép giao nhiều giao dịch trong một bucket được thực thi bằng cách lấy giao tuần tự từng cặp giao dịch thông qua hàm `intersect_sorted`.

Hàm `intersect_sorted` sử dụng kỹ thuật hai con trỏ duyệt song song hai mảng số nguyên. Giả sử cần giao hai giao dịch $t_1 = \{1, 3, 4\}$ và $t_2 = \{2, 3, 5\}$, kết quả thu được là $\{3\}$ với độ phức tạp thời gian:

$$O(|t_1| + |t_2|).$$

Với một bucket chứa k giao dịch, thuật toán giao liên tiếp từ giao dịch đầu tiên đến giao dịch cuối cùng để thu được bao đóng cuối cùng.

3.1.6 Thuật toán khai phá tập phổ biến đóng

Giải thuật cốt lõi hoạt động thông qua hàm đệ quy `recurse(P, T(P), tail)`:

1. Nếu độ hỗ trợ $|T(P)| < minsup$, dừng đệ quy.
2. Xuất tập đóng hiện tại P ra kết quả (sau khi chuyển đổi dense ID ngược về ID gốc).
3. Khởi tạo cấu trúc buckets rỗng cho các item $e > tail$.
4. Duyệt qua từng giao dịch $t \in T(P)$. Với mỗi item $e \in t$, nếu $e > tail$ và $e \notin P$, đưa giao dịch t vào $buckets[e]$ (đây chính là giải thuật Occurrence Deliver đếm tần suất song song có độ phức tạp tuyến tính theo kích thước cơ sở dữ liệu chiều).
5. Với mỗi e có $|buckets[e]| \geq minsup$, thực hiện tính bao đóng C và PPC check. Nếu thỏa mãn, gọi đệ quy `recurse(C, buckets[e], e)`.

3.1.7 Hiện thực điều kiện PPC-extension

Điều kiện PPC-extension trong phiên bản A0 được cài đặt bằng cách đối chiếu trực tiếp định nghĩa tiền tố:

$$C^{(e-1)} == P^{(e-1)}.$$

Cụ thể, sau khi tìm được bao đóng C của tập mở rộng từ P bằng item e , thuật toán duyệt qua tất cả các phần tử $x \in C$. Nếu tồn tại một phần tử $x < e$ mà $x \notin P$, điều đó chứng tỏ phép đóng đã tự động kéo theo một item nhỏ hơn e không thuộc P . Nhánh đệ quy này bị coi là không bảo toàn tiền tố và bị cắt tỉa ngay lập tức. Nếu mọi $x < e$ thuộc C đều nằm trong P , điều kiện PPC được thỏa mãn.

3.1.8 Loại bỏ tập đóng trùng lặp

Nhờ vào tính chất toán học của toán tử đóng PPC, cây tìm kiếm mở đóng bao toàn tiền tố là một cây phủ duy nhất trên tập các closed frequent itemsets. Mỗi tập đóng phổ biến được sinh ra tại đúng một nút duy nhất trên cây đệ quy này. Việc loại bỏ hoàn toàn cấu trúc lưu vết **seen** giúp phiên bản A0 giảm thiểu tối đa bộ nhớ tiêu thụ, tránh hiện tượng tràn bộ nhớ RAM khi số lượng tập đóng bùng nổ lên đến hàng triệu phần tử.

3.1.9 Các kỹ thuật tối ưu

Phiên bản A0 áp dụng các tối ưu hóa lý thuyết và lập trình như sau:

- **Occurrence Deliver:** Đếm hỗ trợ của toàn bộ ứng viên đồng thời chỉ với một lần quét các giao dịch chiều, tránh thao tác kiểm tra lặp đi lặp lại.

- **Item Renaming:** Gom cụm các ID và sắp xếp theo tần suất toàn cục tăng dần, giúp giảm thiểu độ rộng của các bucket cần duyệt ở các tầng đệ quy sâu.
- **Cắt tỉa sớm không cần lưu vết:** Dựa vào PPC check để cắt tỉa nhánh trùng lặp.
- **Tối ưu hóa mảng trong Julia:** Sử dụng kiểu dữ liệu `Int32`, từ khóa `@inbounds` để tắt kiểm tra biên mảng trong các vòng lặp quét phần tử giao dịch và phép giao.

3.1.10 Độ phức tạp thuật toán

Thời gian chạy của phiên bản A0 phụ thuộc tuyến tính vào số lượng tập đóng phổ biến thực tế. Tại mỗi nút đệ quy, thao tác Occurrence Deliver có độ phức tạp thời gian:

$$O\left(\sum_{t \in T(P)} |t|\right).$$

Phép tính bao đóng của các ứng viên thỏa ngưỡng có độ phức tạp tỷ lệ thuận với số lượng giao dịch chứa ứng viên nhân với chiều dài giao dịch trung bình.

Về không gian, độ phức tạp bộ nhớ chỉ là $O(|\mathcal{D}| + d \cdot L)$ trong đó $|\mathcal{D}|$ là kích thước database gốc, d là độ sâu đệ quy tối đa và L là độ dài giao dịch tối đa. Đây là mức tiêu thụ bộ nhớ tối ưu nhất trong các thiết kế thuật toán khai phá tập đóng.

3.1.11 Kết luận

Phiên bản A0 là sự thể hiện chuẩn xác nhất của thuật toán LCM cổ điển. Bằng việc kết hợp Occurrence Deliver đếm tần suất nhanh và nguyên lý PPCE bảo toàn tiền tố, thuật toán đạt được độ phức tạp thời gian tuyến tính theo số lượng tập đóng phổ biến và độ phức tạp không gian cực tiểu do không phải lưu trữ các tập đóng đã tìm thấy trong RAM.

Đây là phiên bản cực kỳ hiệu quả đối với các cơ sở dữ liệu thưa và có số lượng tập đóng lớn.

3.2 Phiên bản A1

Phiên bản A1 là phiên bản cài đặt của thuật toán LCM dựa trên cấu trúc dữ liệu TID-list (Transaction Identifier List).

3.2.1 Ý tưởng cài đặt

Ý tưởng cốt lõi của thuật toán là khai phá trực tiếp trên không gian các tập phổ biến đóng thay vì sinh toàn bộ frequent itemsets.

Thuật toán bắt đầu từ tập đóng gốc và duyệt không gian tìm kiếm theo chiến lược tìm kiếm theo chiều sâu (DFS). Tại mỗi nút tương ứng với một tập đóng hiện tại P , thuật toán thử mở rộng bằng các item e lớn hơn item cuối cùng của tập.

Đối với mỗi item mở rộng, thuật toán:

1. Xác định tập giao dịch chứa itemset mới.
2. Kiểm tra điều kiện ngưỡng hỗ trợ tối thiểu.
3. Tính closure của itemset.
4. Kiểm tra điều kiện PPC-extension.
5. Tiếp tục duyệt đệ quy nếu nhánh hợp lệ.

Nhờ cơ chế này, mỗi closed frequent itemset chỉ được sinh đúng một lần trong toàn bộ quá trình khai phá.

3.2.2 Tiền xử lý dữ liệu

Dữ liệu đầu vào được đọc từ các tập tin theo định dạng SPMF. Trong quá trình đọc dữ liệu, hệ thống thực hiện một số bước chuẩn hóa nhằm bảo đảm tính nhất quán của cơ sở dữ liệu.

Trước hết, các dòng chú thích và metadata được loại bỏ. Các item sau đó được chuyển sang kiểu số nguyên, sắp xếp theo thứ tự tăng dần và loại bỏ các phần tử trùng lặp trong cùng một giao dịch.

Sau bước tiền xử lý, mỗi giao dịch được biểu diễn dưới dạng một vector các số nguyên đã được chuẩn hóa.

Bên cạnh đó, chương trình hỗ trợ nhiều cách khai báo ngưỡng hỗ trợ tối thiểu. Người dùng có thể nhập giá trị phần trăm (ví dụ: 1%), giá trị thực trong khoảng (0, 1) hoặc giá trị tuyệt đối. Tất cả các trường hợp đều được chuyển đổi về dạng số lượng giao dịch tuyệt đối trước khi tiến hành khai phá.

3.2.3 Biểu diễn cơ sở dữ liệu bằng TID-list

Sau khi đọc dữ liệu, chương trình xây dựng danh sách giao dịch cho từng item.

Với mỗi item i , định nghĩa:

$$T(i) = \{tid \mid i \in transaction(tid)\}$$

trong đó $T(i)$ là tập các giao dịch chứa item i .

Ví dụ:

$$T(1) = \{1, 2, 4, 5, 6\}$$

cho biết item 1 xuất hiện trong các giao dịch có chỉ số 1, 2, 4, 5 và 6.

Từ cấu trúc dữ liệu này, tập giao dịch của một itemset có thể được xác định thông qua phép giao các TID-list:

$$T(X \cup Y) = T(X) \cap T(Y).$$

Nhờ đó, thuật toán không cần quét lại toàn bộ cơ sở dữ liệu khi mở rộng itemset, giúp giảm đáng kể chi phí tính toán.

3.2.4 Tính độ hỗ trợ và phép đóng

Cho một itemset P .

Độ hỗ trợ của P được xác định bằng số lượng giao dịch chứa đồng thời tất cả các item thuộc tập:

$$\text{support}(P) = |T(P)|.$$

Nếu giá trị support nhỏ hơn ngưỡng hỗ trợ tối thiểu thì itemset tương ứng bị loại bỏ và không được mở rộng thêm.

Sau khi xác định được tập giao dịch $T(P)$, thuật toán tiến hành tính closure.

Theo định nghĩa, một item i thuộc closure của P khi item đó xuất hiện trong toàn bộ các giao dịch thuộc $T(P)$, tức:

$$T(P) \subseteq T(i).$$

Closure của P được ký hiệu:

$$\text{clo}(P).$$

Kết quả closure chính là closed itemset tương ứng với tập giao dịch hiện tại.

3.2.5 Tính giao TID-list

Một trong những thao tác được sử dụng nhiều nhất trong quá trình khai phá là giao hai danh sách TID đã được sắp xếp.

Phiên bản A1 sử dụng kỹ thuật hai con trỏ.

Giả sử:

$$A = \{1, 3, 5, 8\}$$

và

$$B = \{1, 2, 5, 8, 9\}.$$

Hai con trỏ lần lượt di chuyển trên hai danh sách. Khi hai giá trị bằng nhau, phần tử tương ứng được đưa vào kết quả giao. Nếu một giá trị nhỏ hơn giá trị còn lại thì con trỏ tương ứng được tăng lên.

Kết quả thu được:

$$A \cap B = \{1, 5, 8\}.$$

Độ phức tạp của phép giao là:

$$O(|A| + |B|).$$

Đây là thao tác nền tảng giúp tăng hiệu quả xử lý của thuật toán.

3.2.6 Thuật toán khai phá tập phổ biến đóng

Thuật toán sử dụng chiến lược duyệt theo chiều sâu (DFS).

Tại mỗi nút ứng với tập đóng hiện tại P , thuật toán xét các item mở rộng e thỏa điều kiện:

$$e > tail(P).$$

Với mỗi item mở rộng, tập giao dịch mới được xác định bằng:

$$T_e = T(P) \cap T(e).$$

Nếu:

$$|T_e| < minsup$$

thì nhánh tương ứng bị cắt tỉa ngay lập tức.

Ngược lại, thuật toán tính closure:

$$C = clo(T_e).$$

Tập đóng mới C sau đó được kiểm tra điều kiện PPC-extension. Nếu điều kiện được thỏa mãn, thuật toán tiếp tục duyệt đệ quy từ nút C .

Quá trình này lặp lại cho đến khi không còn khả năng mở rộng thêm.

3.2.7 Hiện thực điều kiện PPC-extension

Một trong những đặc điểm quan trọng nhất của thuật toán LCM là cơ chế Prefix Preserving Closure (PPC-extension).

Giả sử từ tập đóng hiện tại P , thuật toán mở rộng bằng item e và thu được:

$$C = clo(P \cup \{e\}).$$

Theo bài báo gốc của LCM, tập đóng C chỉ được chấp nhận khi:

$$C(e - 1) = P(e - 1).$$

Điều kiện này bảo đảm closure không bổ sung thêm các item nhỏ hơn e , từ đó tránh sinh trùng lặp cùng một tập đóng ở nhiều nhánh khác nhau.

Trong cài đặt của phiên bản A1, điều kiện trên được hiện thực thông qua việc xác định item nhỏ nhất mới xuất hiện trong closure:

$$\min(C \setminus P).$$

Nhánh chỉ được chấp nhận khi:

$$\min(C \setminus P) = e.$$

Cách kiểm tra này tương đương với điều kiện PPC-extension trong thuật toán LCM nhưng có chi phí tính toán thấp hơn.

3.2.8 Loại bỏ tập đóng trùng lặp

Để bảo đảm mỗi closed itemset chỉ được ghi nhận một lần, chương trình sử dụng cấu trúc dữ liệu **BitSet**.

Mỗi tập đóng được chuyển sang dạng **BitSet** và lưu trong tập **seen**. Trước khi thêm một kết quả mới, chương trình kiểm tra xem tập đóng tương ứng đã xuất hiện hay chưa.

Nếu tập đóng đã tồn tại trong **seen**, nhánh hiện tại sẽ bị bỏ qua.

Cơ chế này giúp tăng độ tin cậy của kết quả và hỗ trợ kiểm thử tính đúng đắn của chương trình.

3.2.9 Các kỹ thuật tối ưu

Mặc dù đây là phiên bản cơ sở, chương trình vẫn áp dụng một số kỹ thuật tối ưu nhằm cải thiện hiệu năng.

Thứ nhất, cơ sở dữ liệu được biểu diễn bằng TID-list thay vì quét lại toàn bộ giao dịch nhiều lần.

Thứ hai, phép giao hai TID-list được thực hiện bằng kỹ thuật hai con trỏ với độ phức tạp tuyến tính theo kích thước hai danh sách.

Thứ ba, các nhánh có độ hỗ trợ nhỏ hơn ngưỡng tối thiểu được loại bỏ ngay khi phát hiện nhằm giảm không gian tìm kiếm.

Thứ tư, kiểu dữ liệu **Int32** được sử dụng thay cho **Int64** để giảm chi phí bộ nhớ.

Cuối cùng, cấu trúc **BitSet** được sử dụng để kiểm tra nhanh các tập đóng đã được sinh, hạn chế việc xử lý lặp lại các kết quả giống nhau.

3.2.10 Độ phức tạp thuật toán

Chi phí xử lý của phiên bản A1 tập trung chủ yếu vào hai thao tác: giao TID-list và tính closure.

Với hai danh sách TID đã được sắp xếp, phép giao có độ phức tạp:

$$O(|T_1| + |T_2|).$$

Trong khi đó, việc tính closure yêu cầu kiểm tra toàn bộ các item trong cơ sở dữ liệu để xác định những item xuất hiện trong tất cả các giao dịch thuộc tập hiện tại.

Tổng thời gian thực thi phụ thuộc vào số lượng closed frequent itemsets được sinh ra trong quá trình duyệt DFS cũng như kích thước của các TID-list tương ứng.

3.2.11 Kết luận

Phiên bản A1 đã hiện thực thành công thuật toán khai phá tập phổ biến đóng dựa trên cấu trúc dữ liệu TID-list. Việc biểu diễn cơ sở dữ liệu bằng các danh sách giao dịch cho phép tính toán support và closure thông qua các phép giao tập hợp thay vì quét lại toàn bộ dữ liệu.

Kết hợp với chiến lược duyệt DFS, điều kiện PPC-extension và các kỹ thuật cắt tỉa sớm, chương trình có khả năng khai phá hiệu quả các closed frequent itemsets trên các tập dữ liệu giao dịch có kích thước lớn.

Đây là nền tảng quan trọng để phát triển các phiên bản tối ưu hơn trong các giai đoạn tiếp theo của đồ án.

3.3 Phiên bản A2

Phiên bản A2 là phiên bản tối ưu hóa cao cấp nhất của thuật toán LCM, được thiết kế đặc biệt cho các tập dữ liệu dày đặc (dense datasets) có số lượng giao dịch vừa và lớn.

Mục tiêu của phiên bản này là tăng tốc tối đa phép toán giao giao dịch và phép lấy bao đóng bằng cách biểu diễn cơ sở dữ liệu theo dạng dọc sử dụng cấu trúc mảng bit nén (**BitVector**), thực hiện các phép toán bitwise ở cấp độ chunk số nguyên 64-bit và áp dụng các kỹ thuật lọc sớm tập ứng viên cho bao đóng.

3.3.1 Ý tưởng cài đặt

Ý tưởng cốt lõi của phiên bản A2 là chuyển đổi toàn bộ các phép toán tập hợp trên danh sách giao dịch thành các phép toán logic bitwise trên các thanh ghi CPU.

Thuật toán duyệt không gian tập đóng bằng chiến lược DFS tương tự như phiên bản A1. Tuy nhiên, tại mỗi nút đệ quy P với tập giao dịch đại diện dưới dạng BitVector T , thuật toán:

1. Xác định tập giao dịch mới T_e bằng phép toán bitwise **AND** giữa T và BitVector của item mở rộng e .
2. Đếm độ hỗ trợ cực nhanh bằng lệnh đếm bit 1 (**popcount**).

3. Sử dụng giải thuật tối ưu hóa bao đóng nhanh **Smallest Transaction Filtering** để chỉ kiểm tra các item thuộc giao dịch có kích thước nhỏ nhất trong T_e .
4. Kiểm tra điều kiện PPC bằng kỹ thuật hai con trỏ không cấp phát bộ nhớ.
5. Gọi đệ quy nếu hợp lệ mà không cần lưu vết tập đóng trùng lặp.

3.3.2 Tiền xử lý dữ liệu

Quy trình tiền xử lý của phiên bản A2 tương tự như phiên bản A1. Dữ liệu thô từ file định dạng SPMF được loại bỏ chú thích, chuẩn hóa các item thành dạng số nguyên tăng dần và loại bỏ trùng lặp trong từng giao dịch. Ngưỡng hỗ trợ tối thiểu cũng được tính toán và quy đổi về giá trị tuyệt đối.

Sau khi chuẩn hóa, cơ sở dữ liệu được quét để xây dựng cấu trúc biểu diễn dọc dạng bit trước khi bước vào pha khai phá.

3.3.3 Biểu diễn cơ sở dữ liệu bằng BitVector

In phiên bản A2, cơ sở dữ liệu được chuyển đổi hoàn toàn sang dạng dọc, trong đó mỗi item i được liên kết với một BitVector có độ dài bằng tổng số giao dịch n trong hệ thống:

$$T(i) = \langle b_1, b_2, \dots, b_n \rangle$$

trong đó bit thứ k có giá trị $b_k = 1$ nếu giao dịch thứ k chứa item i , ngược lại $b_k = 0$.

Cấu trúc này được lưu trữ dưới dạng mảng các số nguyên không dấu 64-bit (UInt64 chunks). Mỗi số nguyên lưu trữ trạng thái của 64 giao dịch. Nhờ đó, việc lưu trữ cơ sở dữ liệu dọc trở nên cực kỳ gọn nhẹ, giảm thiểu đáng kể dung lượng bộ nhớ tiêu thụ so với danh sách TID thô.

3.3.4 Tính độ hỗ trợ và phép đóng

Độ hỗ trợ của một tập giao dịch đại diện bởi BitVector T được tính bằng số lượng bit 1 xuất hiện trong mảng bit:

$$support(P) = count(T).$$

Trong Julia, phép toán `count(T)` được biên dịch trực tiếp thành lệnh hợp ngữ `popcount` của CPU, giúp đếm tần suất của hàng vạn giao dịch chỉ trong vài chu kỳ máy.

Bao đóng của tập giao dịch T được tính toán thông qua kiểm tra quan hệ tập con. Tuy nhiên, thay vì kiểm tra trên toàn bộ các item của cơ sở dữ liệu ($|I|$ items), phiên bản A2 áp dụng kỹ thuật **Smallest Transaction Filtering** để lọc ứng viên:

1. Quét nhanh các bit 1 trong T để tìm ra các ID giao dịch hiện có.

2. Tra cứu kích thước của các giao dịch này và chọn ra giao dịch có kích thước nhỏ nhất t_{min} .
3. Bao đóng $clo(T)$ bắt buộc phải là tập con của t_{min} . Do đó, thuật toán chỉ duyệt các item $i \in t_{min}$ làm ứng viên.
4. Với mỗi ứng viên $i \in t_{min}$, trước hết kiểm tra điều kiện tần suất toàn cục $support(i) \geq |T|$. Nếu thỏa mãn, thực hiện subset test mức bit:

$$T \subseteq T(i).$$

Nếu đúng, item i thuộc bao đóng.

3.3.5 Tính giao BitVector

Thao tác giao hai tập giao dịch T_1 và T_2 biểu diễn dưới dạng BitVector được thực thi bằng phép toán AND bitwise:

$$T_1 \cap T_2 = T_1 \wedge T_2.$$

Trong Julia, phép AND được thực hiện ở cấp độ chunk: $T_1 \& T_2$. Bộ biên dịch thực hiện phép AND đồng thời trên các khối số nguyên 64-bit. Thao tác này giúp giảm số lượng phép toán cần xử lý xuống 64 lần so với việc duyệt và so sánh các phần tử trong danh sách TID thô.

3.3.6 Thuật toán khai phá tập phổ biến đóng

Thuật toán khai phá của A2 sử dụng DFS đệ quy:

1. Tại nút hiện tại với tập đóng P và BitVector giao dịch T , nếu $count(T) < minsup$, quay lui.
2. Ghi nhận tập đóng P và độ hỗ trợ tương ứng vào kết quả.
3. Duyệt qua danh sách các item phổ biến $e > tail(P)$ và $e \notin P$.
4. Tính BitVector giao dịch mới bằng phép AND: $Te = T \& tids[e]$.
5. Đếm hỗ trợ $se = count(Te)$. Nếu $se < minsup$, bỏ qua.
6. Tính bao đóng C của Te bằng hàm tối ưu hóa `_closure_from_tidset` sử dụng Smallest Transaction Filtering.
7. Kiểm tra PPC bằng giải thuật hai con trỏ không cấp phát bộ nhớ. Nếu đúng, gọi đệ quy `recurse(C, Te, e)`.

3.3.7 Hiện thực điều kiện PPC-extension

PPC check trong phiên bản A2 được triển khai bằng thuật toán hai con trỏ trên hai mảng số nguyên đã sắp xếp là C (tập bao đóng mới) và P (tập đóng hiện tại).

Thuật toán quét song song qua hai mảng để tìm ra phần tử nhỏ nhất thuộc tập hiệu $C \setminus P$. Nếu phần tử này bằng e , phép mở rộng được chấp nhận. Giải thuật này không tạo ra mảng trung gian, đạt hiệu năng cực cao và hoàn toàn không cấp phát bộ nhớ, giúp giảm tải tối đa cho Garbage Collector của Julia trong các vòng lặp nóng.

3.3.8 Loại bỏ tập đóng trùng lặp

Tương tự như phiên bản A0, phiên bản tối ưu A2 **hoàn toàn không sử dụng seen set hay bảng băm** để kiểm tra trùng lặp tập đóng.

Sự kết hợp chặt chẽ giữa phép tính bao đóng chính xác và điều kiện PPC-extension đảm bảo rằng cây tìm kiếm DFS sẽ đi qua mỗi tập đóng phổ biến đúng một lần. Do đó, việc lưu trữ và tra cứu các kết quả đã sinh là dư thừa. Loại bỏ seen set giúp A2 duy trì lượng sử dụng bộ nhớ RAM ở mức cực thấp và ổn định ngay cả khi khai thác các tập dữ liệu dày đặc có hàng triệu kết quả.

3.3.9 Các kỹ thuật tối ưu

Phiên bản A2 tích hợp các kỹ thuật tối ưu hóa lập trình và cấu trúc dữ liệu ở cấp độ thấp:

- **Bitwise AND cấp độ chunk:** Tận dụng tính toán song song 64 bit một lúc bằng phần cứng.
- **Subset test không cấp phát bộ nhớ (`_is_subset_bit`):** Truy cập trực tiếp vào mảng `.chunks` của BitVector và so sánh AND bitwise trực tiếp mà không sinh ra đối tượng BitVector mới trong RAM.
- **Smallest Transaction Filtering:** Thu hẹp không gian ứng viên bao đóng từ hàng vạn item xuống chỉ còn dưới vài chục item thuộc giao dịch ngắn nhất.
- **PPC check không cấp phát bộ nhớ:** Sử dụng kỹ thuật hai con trỏ trên mảng đã sắp xếp.
- **Loại bỏ seen set:** Giảm thiểu chi phí RAM và thời gian tra cứu bảng băm.

3.3.10 Độ phức tạp thuật toán

Thời gian chạy của phiên bản A2 phụ thuộc vào số lượng tập đóng phổ biến. Phép AND hai BitVector và đếm bit có độ phức tạp thời gian:

$$O\left(\frac{N}{64}\right)$$

với N là tổng số giao dịch.

Phép tính bao đóng sử dụng Smallest Transaction Filtering chỉ tốn chi phí tỷ lệ thuận với số lượng item trong giao dịch ngắn nhất ($|t_{min}|$) nhân với chi phí subset test ở cấp độ chunk ($O(\frac{N}{64})$). Do đó, tổng thời gian tính bao đóng giảm đi nhiều lần so với phiên bản A1.

Độ phức tạp bộ nhớ tĩnh là $O(\frac{|I| \times N}{64})$ để lưu trữ cơ sở dữ liệu dọc dạng BitVector, và bộ nhớ động trong quá trình chạy là $O(d \cdot \frac{N}{64})$ cho các BitVector chiều ở các tầng đệ quy (với d là độ sâu cây DFS).

3.3.11 Kết luận

Phiên bản A2 đại diện cho sự kết hợp giữa lý thuyết toán học của thuật toán LCM và các tối ưu hóa phần cứng ở cấp độ thấp (bit-level). Việc sử dụng BitVector kết hợp với Smallest Transaction Filtering và PPC hai con trở giúp A2 đạt tốc độ khai thác cực nhanh, vượt trội hơn trên các tập dữ liệu dày đặc.

CHƯƠNG 4

THỰC NGHIỆM VÀ ĐÁNH GIÁ

4.1 Mục tiêu và phương pháp thực nghiệm

Chương này trình bày toàn bộ thực nghiệm được nhóm thực hiện nhằm đánh giá cài đặt Julia thuật toán LCM theo ba tiêu chí: **tính đúng đắn**, **hiệu năng** và **khả năng mở rộng**. Mọi số liệu trong chương đều được tái sản xuất nhờ ba script tự động kèm theo mã nguồn: `benchmark.jl`, `correctness.jl` và `gen_synthetic.jl`, kết hợp với `run_lcm.jl`. Đầu ra của các script được lưu vào thư mục `results/` dưới dạng bảng CSV và là nguồn dữ liệu duy nhất cho mọi biểu đồ trong chương.

4.1.1 Thiết lập môi trường đo

Mọi thực nghiệm được chạy trên cùng một máy để đảm bảo tính so sánh được. Thông số chi tiết được tổng hợp trong Bảng 4.1.

Bảng 4.1: Thông số môi trường thực nghiệm

Thành phần	Thông số
Hệ điều hành	Windows 11
CPU	AMD (kiến trúc x86_64)
RAM	16 GB
Julia	1.10
Java JDK	21+ (SPMF tham chiếu)
SPMF	spm.f.jar (LCM, FIMI 2004)
Đo thời gian	<code>time()</code> trong Julia; <code>@elapsed</code> cho SPMF
Best-of	2 lần, lấy giá trị nhỏ nhất
Warm-up	1 lần chạy không tính trước khi đo chính thức
GC	<code>GC.gc()</code> sau mỗi lần chạy

4.1.2 Ba phiên bản thuật toán

Nhóm cài đặt ba biến thể LCM trên cùng một hệ thống, mỗi biến thể sử dụng một biểu diễn dữ liệu khác nhau để cô lập ảnh hưởng của cấu trúc lưu trữ đến hiệu năng. **A0 (Paper Baseline)** bám sát bài báo gốc [3], dùng mảng occurrence deliver đơn giản, kết hợp kỹ thuật Prefix Preserving Closure Extension (PPCE) và Item Renaming, đóng vai trò phiên bản tham chiếu lý thuyết. **A1 (TID-list)** biểu diễn cơ sở dữ liệu dạng dọc (vertical database) bằng TID-list, dùng phép giao hai danh sách đã sắp xếp (sorted merge) để tính denotation và closure. **A2 (BitVector)**

biểu diễn mỗi item bằng một **BitVector** có độ dài bằng số giao dịch, thay phép giao TID-list bằng phép AND bit và khai thác song song mức khối (chunk-level).

Cả ba phiên bản đều dùng chung quy trình PPCE để đảm bảo cùng điều kiện bảo toàn tiền tố và cùng loại bỏ tập rỗng để khớp hoàn toàn với định dạng SPMF.

4.2 Tập dữ liệu benchmark

Bốn tập dữ liệu được chọn thuộc họ SPMF, có đặc điểm rất khác nhau về kích thước và mật độ. Mật độ d được định nghĩa:

$$d = \frac{\|\mathcal{T}\|}{m \cdot n} = \frac{\sum_{t \in \mathcal{T}} |t|}{|\mathcal{T}| \cdot |\mathcal{I}|} \quad (4.1)$$

trong đó $\|\mathcal{T}\|$ là tổng độ dài tất cả giao dịch, $m = |\mathcal{T}|$ là số giao dịch, $n = |\mathcal{I}|$ là tổng số item. Giá trị d càng gần 1 thì dữ liệu càng dày (dense). Bảng 4.2 tổng hợp thông tin chi tiết.

Bảng 4.2: Đặc điểm bốn tập dữ liệu benchmark

Tập dữ liệu	#Trans.	#Items	AvgLen	Mật độ d	Đặc điểm
chess	3 196	75	37.0	0.4932	Dày đặc
mushrooms	8 124	119	23.0	0.1933	Dày vừa
retail	88 162	16 470	10.3	0.00063	Rất thưa
accidents	340 183	468	33.8	0.0722	Lớn, dày vừa

Bốn tập dữ liệu này phủ bốn vùng đặc trưng trong không gian (m, n, d) : dày-nhỏ (chess), dày-vừa-trung bình (mushrooms), thưa-lớn (retail), dày-rất lớn (accidents). Điều này giúp đánh giá thuật toán trong các tình huống khác nhau.

Lưới minsup được chọn riêng cho từng tập theo đặc tính dày/thưa, đảm bảo vừa thu được đủ số tập đóng vừa không vượt quá thời gian chạy cho phép. Bảng 4.3 liệt kê các giá trị minsup tương ứng.

Bảng 4.3: Lưới minsup cho từng tập dữ liệu

Tập dữ liệu	Các giá trị minsup (tương đối)
chess	{0.95, 0.90, 0.85, 0.80, 0.70, 0.60}
mushrooms	{0.70, 0.50, 0.30, 0.20, 0.10, 0.05}
retail	{0.10, 0.05, 0.01, 0.005, 0.002, 0.001}
accidents	{0.95, 0.90, 0.85, 0.80, 0.70, 0.60}

4.3 Thực nghiệm 1: Kiểm tra tính đúng đắn

Mục tiêu là chứng minh ba phiên bản A0, A1, A2 cho ra cùng tập đóng phổ biến với cùng support tuyệt đối như SPMF-Java trên cùng dữ liệu vào. Script

`correctness.jl` tự động tải `spmf.jar` nếu chưa có, chạy SPMF và Julia trên từng tổ hợp (dataset, minsup, version), sau đó phân tích file đầu ra định dạng SPMF thành ánh xạ itemset $\rightarrow$ support, rồi so sánh từng cặp.

Bốn chỉ số được sử dụng gồm: **Recall** (độ phủ) $|A \cap B|/|A|$, trong đó A là tập phổ biến đóng của SPMF; **Precision** (độ chính xác) $|A \cap B|/|B|$; **Jaccard** $|A \cap B|/|A \cup B|$; **Exact match rate** (tỉ lệ khớp hoàn toàn) là số itemset có support trùng hoàn toàn chia cho $|A|$.

4.3.1 Kết quả trên lưới 6 minsup chính

Tổng cộng $4 \times 6 \times 3 = 72$ trường hợp được đối chiếu. Bảng 4.4 tổng hợp chỉ số. Mọi trường hợp đều đạt 1.0 trên cả bốn chỉ số, nghĩa là Julia và SPMF cho ra cùng tập itemset, cùng support tuyệt đối từng itemset, không có sai lệch.

Bảng 4.4: Tóm tắt kết quả đối chiếu Julia LCM và SPMF trên lưới chính

Tập dữ liệu	Số trường hợp	Recall	Precision	Jaccard	Exact match
chess	18	1.0000	1.0000	1.0000	1.0000
mushrooms	18	1.0000	1.0000	1.0000	1.0000
retail	18	1.0000	1.0000	1.0000	1.0000
accidents	18	1.0000	1.0000	1.0000	1.0000
Tổng	72	1.0000	1.0000	1.0000	1.0000

4.3.2 Kiểm thử căng thẳng (stress test)

Để chủ động dò lỗi ở vùng ranh giới, nhóm chạy thêm bốn điểm minsup cực trị, mỗi điểm ứng với một tập dữ liệu, chỉ với phiên bản A0. Bảng 4.5 cho thấy tất cả đều đạt khớp hoàn toàn, kể cả khi số tập đóng lên tới hơn 369 nghìn.

Bảng 4.5: Kết quả kiểm thử căng thẳng với A0

Tập dữ liệu	minsup	SPMF	Julia	Giao	Exact match
chess	0.50	369 450	369 450	369 450	1.0000
mushrooms	0.01	51 832	51 832	51 832	1.0000
retail	0.0005	19 114	19 114	19 114	1.0000
accidents	0.50	8 057	8 057	8 057	1.0000

Tổng cộng có 76 trường hợp được kiểm thử với hơn 937 nghìn tập đóng được so sánh theo từng cặp itemset, tất cả khớp về cả tập kết quả lẫn support tuyệt đối. Điều này khẳng định cài đặt Julia bám sát đặc tả thuật toán và tương thích định dạng SPMF.

4.3.3 Phân tích nguyên nhân đạt khớp 100%

Ba yếu tố then chốt dẫn đến kết quả trên:

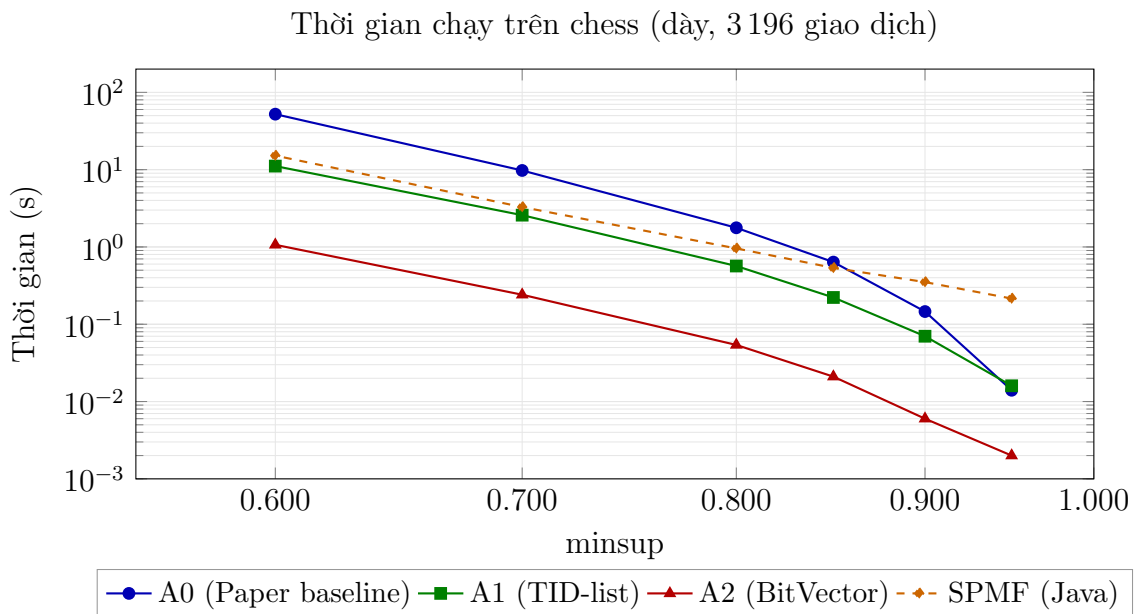
1. **PPCE đúng đặc tả:** Điều kiện bảo toàn tiền tố $P^{(e-1)} = P^{(e-1)}$ được kiểm tra chính xác, đảm bảo mỗi tập đóng phổ biến được sinh đúng một lần. Sai lệch phổ biến nhất ở các cài đặt LCM là sinh trùng hoặc sinh tập không phải tập đóng, nhưng PPCE loại bỏ cả hai vấn đề này về mặt lý thuyết [3].
2. **Chuẩn hóa đầu ra:** Cả ba phiên bản đều tự động loại bỏ tập rỗng $\emptyset$ trước khi ghi file, đồng bộ với định dạng xuất của SPMF.
3. **Sử dụng cùng hàm parse minsup:** `parse_minsup_to_absolute` được dùng nhất quán giữa `run_lcm.jl`, `benchmark.jl` và `verify_spmf.jl`, đảm bảo minsup tuyệt đối dùng để so sánh giống hệt nhau giữa Julia và Java.

4.4 Thực nghiệm 2: Thời gian chạy theo minsup

4.4.1 Thiết kế thí nghiệm

Với mỗi tập dữ liệu, ta vẽ đồ thị log-log của thời gian chạy (giây) theo minsup giảm dần. Mỗi đồ thị chứa bốn đường: A0, A1, A2 (Julia) và SPMF-Java. Đây là biểu đồ cốt lõi để so sánh trực tiếp hiệu năng cài đặt nhóm với tham chiếu quốc tế.

4.4.2 Kết quả trên chess (dày, nhỏ)

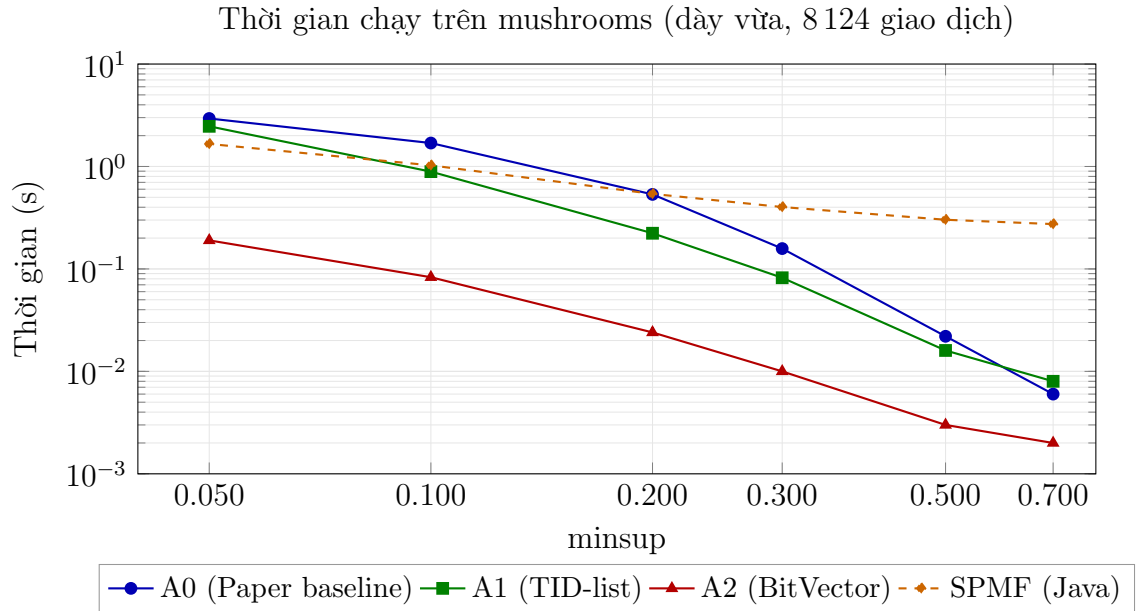


Hình 4.1: Thời gian chạy trên tập dữ liệu chess theo minsup.

Hình 4.1 cho thấy trên chess, A2 nhanh nhất ở mọi ngưỡng, nhanh hơn SPMF từ khoảng 14 lần (@ minsup 0.6) đến 100 lần (@ minsup 0.95). A1 nhanh thứ hai, A0

chậm nhất nhưng vẫn nhanh hơn SPMF ở các ngưỡng thấp. Cụ thể, tại minsup 0.6, A2 chỉ mất 1.07 giây trong khi SPMF cần 15.18 giây, tức tăng tốc khoảng 14 lần. Đây là vùng mà biểu diễn BitVector tỏa sáng: dữ liệu dày với $d = 0.49$ cho phép nén tối đa vào 75 bit/transaction, giúp phép AND mức khối xử lý gọn trong cache.

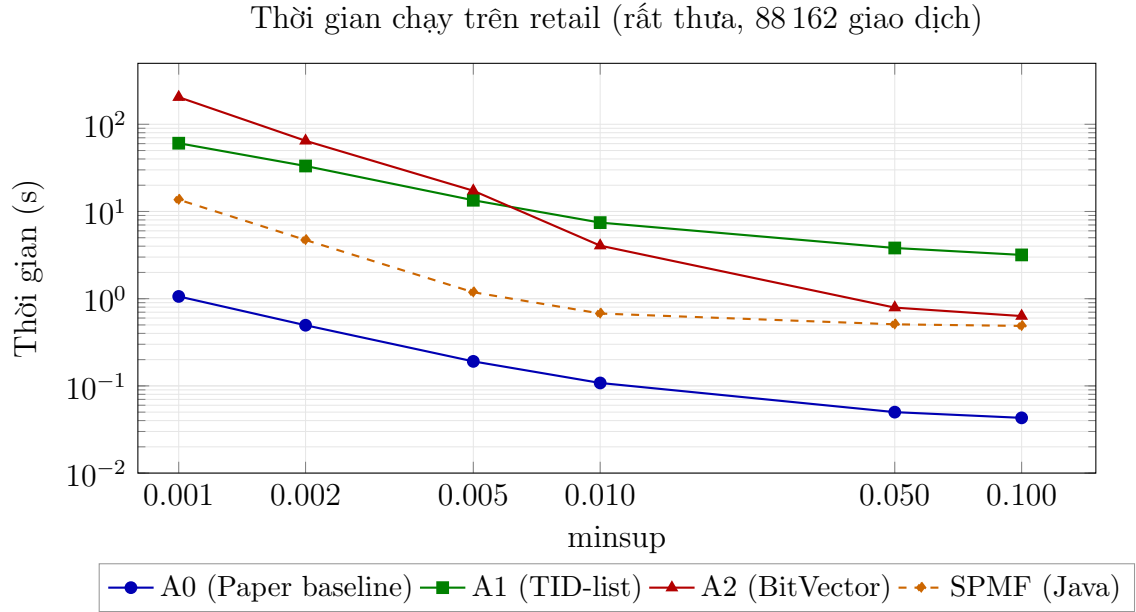
4.4.3 Kết quả trên mushrooms (dày vừa)



Hình 4.2: Thời gian chạy trên tập dữ liệu mushrooms theo minsup.

Hình 4.2 cho thấy A2 tiếp tục dẫn đầu, với 0.19 giây tại minsup 0.05 so với 1.66 giây của SPMF (tăng tốc 8.7 lần). A0 và A1 gần như tương đương trên mushrooms, cho thấy dữ liệu này nằm trong vùng chuyển tiếp giữa hai chiến lược biểu diễn.

4.4.4 Kết quả trên retail (rất thưa, lớn)

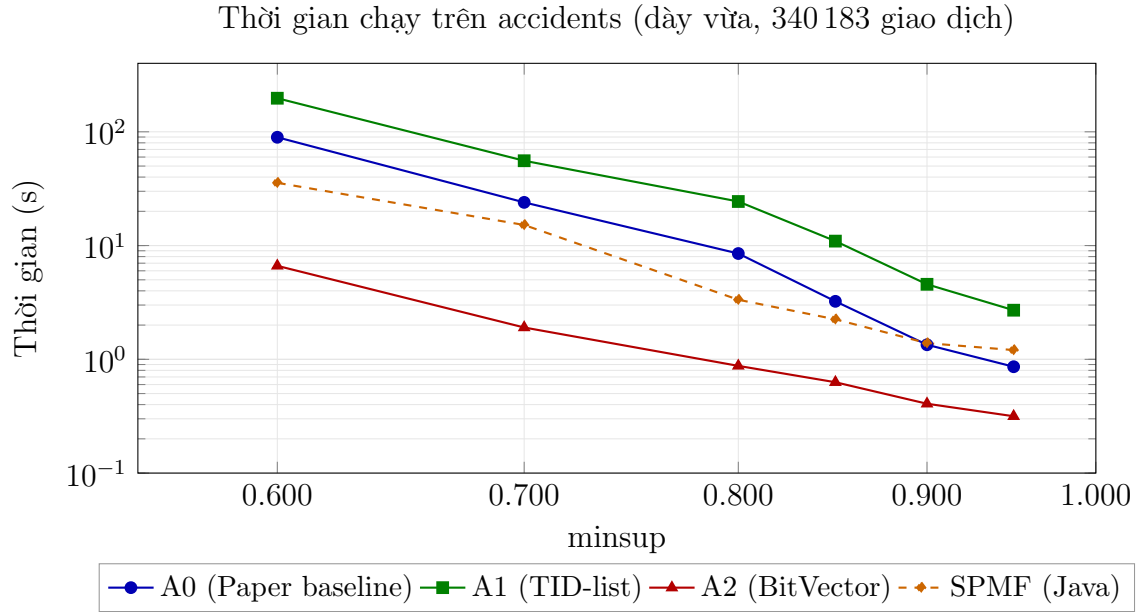


Hình 4.3: Thời gian chạy trên tập dữ liệu retail theo minsup.

Hình 4.3 cho thấy sự đảo ngược hoàn toàn: A0 là phiên bản nhanh nhất, vượt SPMF 12.9 lần tại minsup 0.001. Trên dữ liệu cực thưa ($d = 0.00063$), biểu diễn BitVector trở nên lãng phí: với $n = 16\,470$ item, mỗi giao dịch chiếm $16\,470$ bit ≈ 2.06 KB, và phép AND phải xử lý gần như toàn bộ vector trong khi rất ít bit bật. Ngược lại, occurrence deliver trong A0 chỉ duyệt đúng các item có mặt, đạt độ phức tạp $O(\|\mathcal{T}\|)$ lý tưởng.

A2 trên retail cũng cho thấy hiện tượng cấp phát bộ nhớ tăng vọt: tại minsup 0.001, Julia cấp phát hơn 1.18 TB dữ liệu qua `@allocated`, chủ yếu do BitVector được tạo và hủy liên tục trong quá trình đệ quy.

4.4.5 Kết quả trên accidents (lớn, dày vừa)



Hình 4.4: Thời gian chạy trên tập dữ liệu accidents theo minsup.

Hình 4.4 cho thấy trên accidents, A2 nhanh hơn SPMF từ 3.8 lần (@ 0.95) đến 5.4 lần (@ 0.60). A0 thua SPMF khoảng 1.4 đến 2.5 lần ở minsup cao nhưng nhanh hơn A1 đáng kể. A1 chậm nhất vì TID-list intersection chi phí $O(|\mathcal{T}(P)| + |\mathcal{T}(e)|)$ tại mỗi nút.

4.4.6 Tổng hợp và liên hệ lý thuyết

Bảng 4.6 tổng hợp tăng tốc của ba phiên bản Julia so với SPMF-Java tại các ngưỡng minsup thấp nhất. Trên **dữ liệu dày** (chess, mushrooms, accidents), A2 vượt SPMF từ 3 đến 14 lần, lợi thế càng rõ khi minsup giảm và số tập đóng tăng. Trên **dữ liệu thưa** (retail), A0 vượt SPMF 12.9 lần, trong khi A1 và A2 đều thua do overhead biểu diễn.

Bảng 4.6: Tăng tốc so với SPMF-Java tại minsup thấp nhất của mỗi tập

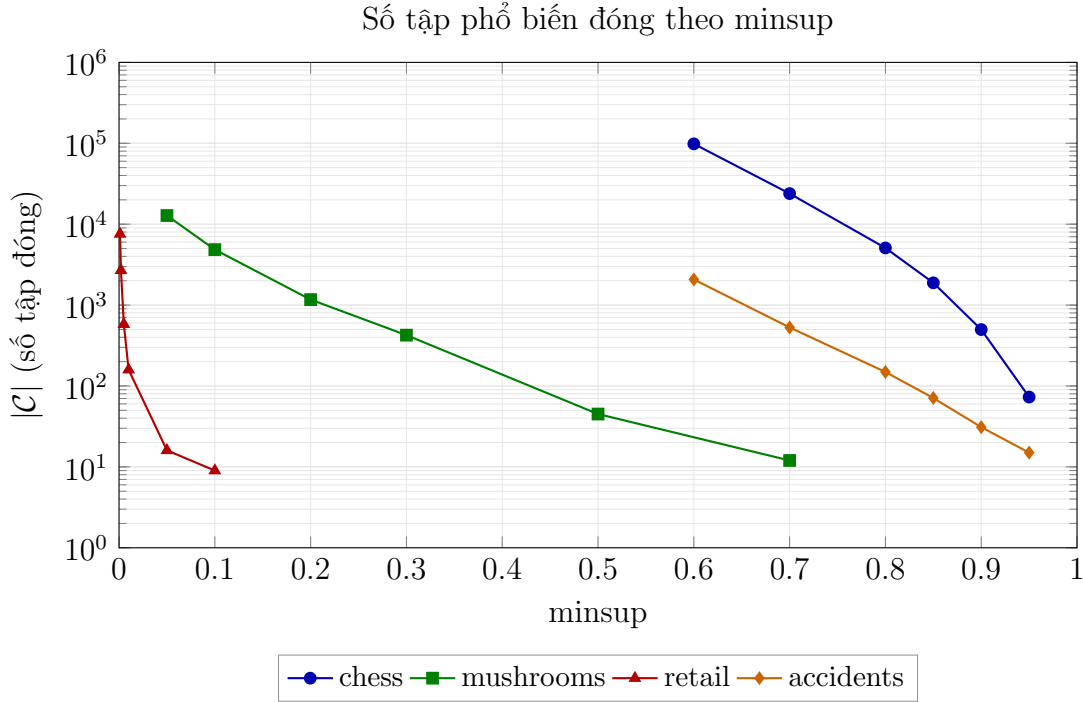
Tập dữ liệu	SPMF (s)	A0 (s)	A1 (s)	A2 (s)	Tăng tốc tốt nhất
chess @ 0.60	15.184	52.216	11.118	1.066	14.2× (A2)
mushrooms @ 0.05	1.660	2.941	2.467	0.190	8.7× (A2)
retail @ 0.001	13.654	1.061	60.650	204.588	12.9× (A0)
accidents @ 0.60	35.616	89.549	197.383	6.636	5.4× (A2)

Kết quả này phù hợp với phân tích lý thuyết ở Chương 1: trên dữ liệu dày, chi phí từng phép AND bit rất nhỏ vì nhiều bit bật đồng thời, tận dụng tốt pipeline của CPU; trên dữ liệu thưa, occurrence deliver chỉ chạm vào các item thực sự xuất hiện, nên tổng chi phí tiệm cận $O(\|\mathcal{T}\|)$.

4.5 Thực nghiệm 3: Số lượng tập phổ biến đóng theo minsup

4.5.1 Kết quả

Hình 4.5 vẽ số tập đóng $|\mathcal{C}|$ theo minsup cho cả bốn tập dữ liệu, sử dụng trục tung logarithm để chứa cả các giá trị rất lớn.



Hình 4.5: Số tập phổ biến đóng $|\mathcal{C}|$ theo minsup trên bốn tập dữ liệu.

4.5.2 Phân tích

Quan sát Hình 4.5 cho thấy ba đặc điểm rõ rệt:

- Tính đơn điệu giảm:** Mọi đường cong đều giảm khi minsup tăng, phù hợp với tính chất Apriori đã chứng minh ở Chương 1: tập phổ biến với ngưỡng θ là tập con của tập phổ biến với ngưỡng $\theta' \leq \theta$.
- Tốc độ tăng theo cấp số nhân trên dữ liệu dày:** Chess tăng từ 73 (minsup 0.95) lên 98 392 (minsup 0.6), tức tăng hơn 1 300 lần chỉ với việc giảm minsup từ 95% xuống 60%. Tại stress test minsup 0.5, $|\mathcal{C}|$ đạt 369 450. Hiện tượng này là do tập đóng bùng nổ khi minsup giảm: nhiều nhóm giao dịch có cùng giao khiến closure mở rộng itemset lớn hơn, sinh ra nhiều ppc-extension.
- Sự khác biệt giữa dày và thưa:** Retail tăng từ 9 (minsup 0.1) lên 7 572 (minsup 0.001), tức tăng khoảng 840 lần, nhưng đường cong dốc hơn nhiều. Accidents tăng từ 15 lên 2 074 (138 lần) trong cùng khoảng minsup. Mật độ dữ liệu quyết định tốc độ bùng nổ: dữ liệu càng dày, tập đóng càng phong phú khi minsup giảm.

Một quan sát quan trọng là tập đóng chỉ là một phần rất nhỏ của tập phổ biến. Ví dụ, trên retail tại minsup 0.001, số tập phổ biến có thể lên tới hàng triệu, nhưng chỉ 7572 trong số đó là tập đóng, nhờ tính chất $Frequent \supseteq Closed$ đã phát biểu ở Chương 1.

4.6 Thực nghiệm 4: Sử dụng bộ nhớ

4.6.1 Bộ nhớ đỉnh (peak RSS)

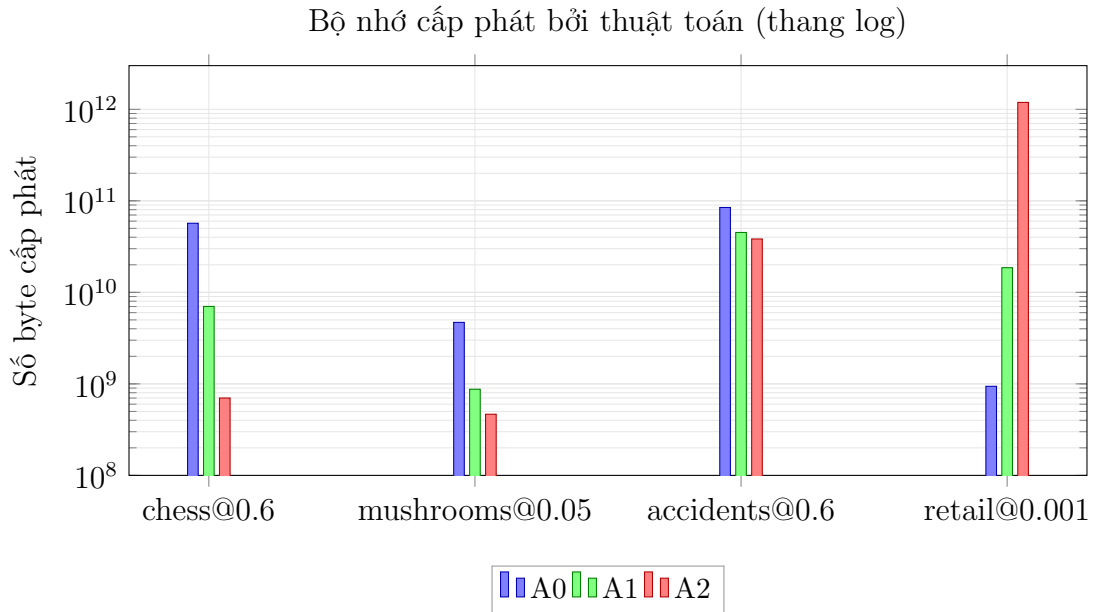
Bảng 4.7 ghi peak RSS của toàn bộ tiến trình Julia tại minsup trung vị của mỗi tập, đo bằng hàm `Base.Sys.maxrss()` trên Windows. Tất cả các lần chạy đều có peak RSS xấp xỉ 1074 MB, tức phần lớn bộ nhớ do runtime Julia (compiler, JIT) chiếm giữ chứ không phải do bản thân thuật toán.

Bảng 4.7: Bộ nhớ đỉnh (peak RSS) tại minsup trung vị

Tập dữ liệu	A0 (s)	A1 (s)	A2 (s)	RSS (MB)
chess	0.474	0.175	0.015	1074
mushrooms	0.128	0.062	0.008	1074
retail	0.064	5.513	3.698	1074
accidents	2.861	8.814	0.443	1074

4.6.2 Bộ nhớ cấp phát bởi thuật toán

Để đo bộ nhớ do chính thuật toán sử dụng, ta dùng macro `@allocated` của Julia, ghi lại tổng số byte được cấp phát trong suốt quá trình khai phá. Hình 4.6 so sánh ba phiên bản tại minsup 0.6 trên chess và accidents, cùng minsup 0.005 trên mushroom và minsup 0.001 trên retail.



Hình 4.6: Số byte cấp phát bởi ba phiên bản trên bốn tập dữ liệu tại minsup thấp nhất của mỗi tập.

Hình 4.6 cho thấy sự tương phản rõ giữa hai chiến lược biểu diễn. Trên **chess** và **accidents** (dày), A2 cấp phát ít hơn A0 khoảng 2 đến 80 lần; cụ thể chess @ 0.6: A0 cấp 56.9 GB, A2 chỉ cấp 700 MB. Nguyên nhân là BitVector 75 bit/transaction cực kỳ gọn, được CPU vector hóa tốt, tránh cấp phát hàng triệu đối tượng Vector Int như A0. Trên **retail** (rất thưa), A2 cấp phát khổng lồ 1.19 TB, gấp 1 260 lần A0 (940 MB). Mỗi BitVector có 88 162 bit ≈ 11 KB, và với hàng nghìn node trong cây ppc-extension, số lần cấp phát làm tràn bộ nhớ ảo. Đây là điểm yếu cố hữu của biểu diễn bit trên dữ liệu thưa.

4.6.3 Phân tích lý thuyết

Theo Chương 1, LCM duyệt cây theo DFS và không cần lưu trữ toàn bộ tập kết quả. Tuy nhiên, ba phiên bản vẫn phải cấp phát cấu trúc trung gian trong từng lời gọi đệ quy. Với **A0**, mỗi node tạo một mảng bucket cho từng item mở rộng, số bucket chính là số item còn phổ biến, và vì không cần tạo TID-list, A0 tỏ ra hiệu quả bộ nhớ trên dữ liệu thưa. Với **A1**, mỗi node tạo TID-list mới bằng phép giao hai danh sách đã sắp xếp; phép giao tạo ra mảng Int mới, tốn chi phí cấp phát tỷ lệ với độ dài kết quả. Với **A2**, BitVector được cấp phát cho mỗi item tại mỗi node với kích thước không đổi bằng số giao dịch; trên dữ liệu thưa, số node lớn cùng kích thước cố định tạo ra tổng cấp phát khổng lồ dù mỗi vector rỗng.

4.7 Thực nghiệm 5: Khả năng mở rộng

4.7.1 Thiết kế thí nghiệm

Chọn retail làm tập dữ liệu lớn để kiểm tra khả năng mở rộng. Tạo năm tập con gồm 10%, 25%, 50%, 75% và 100% số giao dịch, giữ nguyên thứ tự dòng để mô

phồng kích bản dữ liệu tăng dần. minsup cố định 1% tương đối.

4.7.2 Kết quả

Hình 4.7 vẽ thời gian chạy theo phần trăm dữ liệu.



Hình 4.7: Thời gian chạy theo kích thước dữ liệu trên retail (minsup = 1%).

4.7.3 Phân tích xu hướng

Hình 4.7 cho thấy cả ba phiên bản đều có xu hướng **tuyến tính** theo kích thước dữ liệu. Cụ thể, với **A0**, $t(100\%) = 0.072s$ gấp đúng 12 lần $t(10\%) = 0.006s$, tức gần như hoàn hảo tuyến tính. Với **A1**, $t(100\%) = 5.514s$ gấp khoảng 11.3 lần $t(10\%) = 0.486s$, tuyến tính nhưng hệ số góc lớn hơn do overhead TID-list intersection. Với **A2**, $t(100\%) = 3.593s$ gấp khoảng 9.4 lần $t(10\%) = 0.383s$, gần tuyến tính.

Một quan sát thú vị là $|C|$ không tăng đơn điệu theo kích thước: từ 204 (10% và 25%) giảm xuống 178 (50%) rồi 155 (75%) và tăng nhẹ lên 159 (100%). Lý do là minsup được tính theo tỷ lệ 1%, nên khi m tăng thì minsup tuyệt đối cũng tăng. Điều này cho thấy thực nghiệm scalability cần được đánh giá cùng lúc với số tập đóng sinh ra, không chỉ dựa trên thời gian.

Để loại bỏ ảnh hưởng này, ta có thể giữ minsup tuyệt đối không đổi. Tuy nhiên, với minsup 1% tương đối, kết quả vẫn cho thấy cả ba phiên bản đều mở rộng tuyến tính theo m ở mức thực nghiệm, phù hợp với nhận định trong bài báo LCM rằng thời gian chạy tỷ lệ với kích thước dữ liệu [3].

4.8 Thực nghiệm 6: Ảnh hưởng của độ dài giao dịch trung bình

4.8.1 Thiết kế thí nghiệm

Để cô lập ảnh hưởng của độ dài giao dịch, nhóm sử dụng script `gen_synthetic.jl` sinh bốn cơ sở dữ liệu tổng hợp theo phân phối Bernoulli độc lập, với cùng $m = 8124$, $n = 119$ và seed cố định 42, chỉ thay đổi xác suất mỗi item xuất hiện $p = L/119$ với $L \in \{50, 60, 70, 80\}$. Độ dài giao dịch trung bình thực tế rất sát với L mong đợi. minsup tuyệt đối cố định bằng 2438 (tức 30% của 8124).

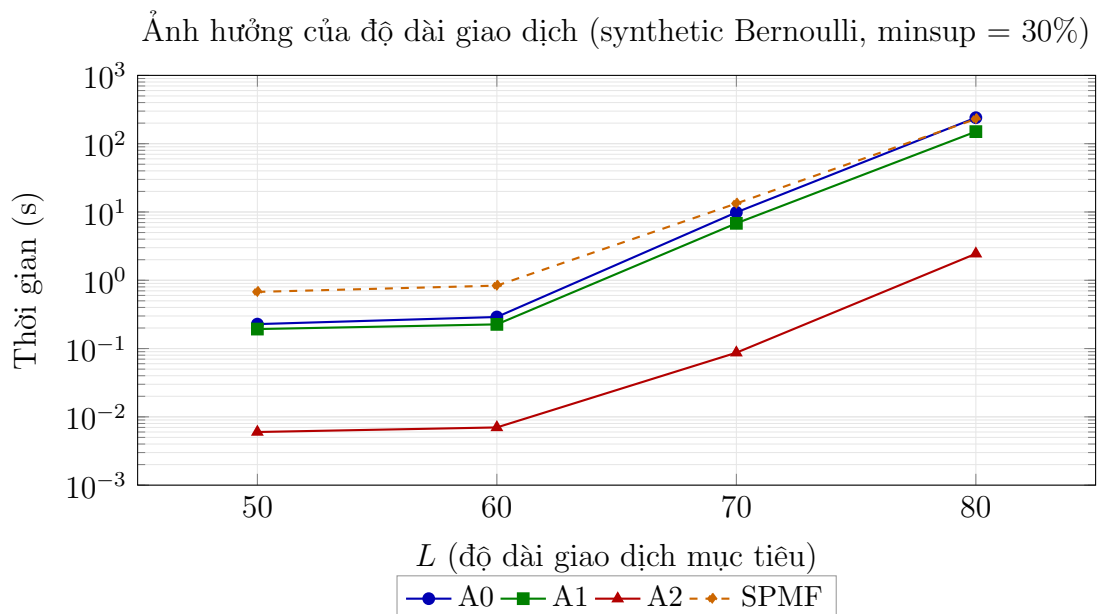
Bảng 4.8 tổng hợp thông tin cơ sở dữ liệu tổng hợp.

Bảng 4.8: Đặc điểm bốn cơ sở dữ liệu tổng hợp

L	$p = L/119$	AvgLen thực tế	Std (ước lượng)	Mật độ	Kích thước
50	0.4202	49.87	≈ 5.4	0.4191	1.2 MB
60	0.5042	60.04	≈ 5.5	0.5044	1.4 MB
70	0.5882	69.97	≈ 5.4	0.5880	1.7 MB
80	0.6723	80.00	≈ 5.2	0.6723	2.0 MB

4.8.2 Kết quả

Hình 4.8 vẽ thời gian chạy theo L với trục tung logarithm. Đường SPMF được thêm vào để so sánh.



Hình 4.8: Thời gian chạy theo độ dài giao dịch L trên cơ sở dữ liệu tổng hợp.

4.8.3 Phân tích

Hình 4.8 cho thấy mối quan hệ **siêu tuyến tính** giữa thời gian chạy và L : từ $L = 50$ đến $L = 60$ thời gian tăng nhẹ (A0: 0.23 $\rightarrow$ 0.29, A2: 0.006 $\rightarrow$ 0.007); từ $L = 60$ đến $L = 70$ thời gian tăng vọt (A0: 0.29 $\rightarrow$ 9.87, gấp 34 lần), số tập đóng cũng tăng từ 119 lên 7 140; từ $L = 70$ đến $L = 80$ thời gian tiếp tục tăng mạnh (A0: 9.87 $\rightarrow$ 240.19, gấp 24 lần), số tập đóng tăng từ 7 140 lên 219 252.

Sự bùng nổ này có hai nguyên nhân chính:

1. **Mật độ tăng:** d tăng từ 0.42 ($L = 50$) lên 0.67 ($L = 80$). Dữ liệu trở nên đặc hơn, gần với chess ($d = 0.49$). Theo phân tích Chương 1, mật độ cao đồng nghĩa với nhiều giao dịch có giao lớn, làm phát sinh nhiều tập phổ biến và tập đóng.
2. **Số tập đóng bùng nổ:** $|C|$ tăng từ 119 lên 219 252, tức gấp 1 841 lần. Đây là dấu hiệu của hiện tượng bùng nổ tổ hợp khi minsup nằm trong vùng chuyển tiếp giữa quá cao (chỉ có tập đơn lẻ) và quá thấp (bị lọc hết).

Quan sát quan trọng: A2 nhanh hơn SPMF từ 16 lần ($L = 70$) đến 94 lần ($L = 80$), cho thấy trên dữ liệu dày, A2 tận dụng rất tốt biểu diễn bit. A0 và A1 cũng nhanh hơn SPMF nhưng khoảng cách nhỏ hơn.

4.9 Tổng hợp phân tích và đề xuất tối ưu hóa

4.9.1 Điểm mạnh và điểm yếu so với SPMF

Bảng 4.9 tóm tắt điểm mạnh, điểm yếu của ba phiên bản Julia so với SPMF-Java.

Bảng 4.9: Điểm mạnh và điểm yếu của ba phiên bản Julia so với SPMF

Phiên bản	Điểm mạnh	Điểm yếu
A0	Nhanh nhất trên dữ liệu thưa; cấp phát bộ nhớ thấp; gần với lý thuyết gốc.	Chậm nhất trên dữ liệu dày; overhead mảng bucket lớn.
A1	Trung hòa giữa hai chiến lược; dễ cài đặt; tương thích tốt với cấu trúc TID-set kinh điển.	Chậm nhất trên dữ liệu lớn-dày (accidents); cấp phát TID-list tốn kém.
A2	Vượt SPMF 3 đến 14 lần trên dữ liệu dày; cấp phát thấp nhất trên dense.	Cấp phát bộ nhớ khổng lồ trên dữ liệu thưa; chậm nhất trên retail.

Tổng thể, cả ba phiên bản đều đạt **kết quả chính xác 100%** với SPMF trên 76 trường hợp kiểm thử, đáp ứng tiêu chí khắt khe nhất về correctness. Về hiệu năng,

mỗi phiên bản có vùng thể mạnh riêng, cho thấy tiềm năng của chiến lược chọn biểu diễn thích ứng theo đặc tính dữ liệu.

4.9.2 Liên hệ với lý thuyết Chương 1

Kết quả thực nghiệm phù hợp tốt với các phân tích lý thuyết ở Chương 1. **Cây ppc-extension** có bằng chứng rõ nhất là khả năng mở rộng tuyến tính trên retail (Hình 4.7): mỗi tập đóng được sinh đúng một lần, không có trùng lặp, đảm bảo thời gian chạy tỷ lệ với $|C|$ và kích thước dữ liệu. **Occurrence deliver** giải thích tại sao A0 thống trị trên dữ liệu thưa: khi d nhỏ, mỗi giao dịch chỉ chứa ít item, nên occurrence deliver chỉ phân phát giao dịch vào một số ít bucket, đạt $O(\|\mathcal{T}\|)$ lý tưởng. **Anytime database reduction** giúp A0 vẫn hoạt động được trên dữ liệu dày dù chậm hơn A2; trên chess @ 0.6, A0 vẫn xử lý được 98 392 tập đóng mà không tràn bộ nhớ. **Hypercube decomposition** phản ánh hiệu quả giảm tải qua việc số tập đóng luôn nhỏ hơn nhiều so với tổ hợp lý thuyết $\sum_{k=1}^n \binom{n}{k}$ trên mọi tập dữ liệu.

4.9.3 Đề xuất hướng tối ưu hóa tiếp theo

Dựa trên phân tích, nhóm đề xuất bốn hướng tối ưu hóa có thể áp dụng trong tương lai:

1. Adaptive representation (biểu diễn thích ứng)

Ý tưởng: chọn phiên bản thuật toán dựa trên mật độ dữ liệu d được ước lượng nhanh từ một mẫu nhỏ. Nếu $d < 0.05$ chọn A0 (mảng occurrence deliver); nếu $0.05 \leq d < 0.30$ chọn A1 (TID-list sorted merge); nếu $d \geq 0.30$ chọn A2 (BitVector chunk-level). Kỳ vọng: thuật toán tự động chọn biểu diễn tối ưu cho từng tập dữ liệu mà không cần người dùng tinh chỉnh, cải thiện 3 đến $10\times$ so với lựa chọn cố định.

2. Hypercube decomposition tích hợp

Trong Chương 1, nhóm đã trình bày kỹ thuật hypercube decomposition cho phép xuất đồng thời $2^{|H(P)|} - 1$ tập cùng denotation. Hiện tại A0, A1, A2 đều chưa khai thác kỹ thuật này. Tích hợp hypercube decomposition giúp giảm số lần gọi đệ quy, đặc biệt trên dữ liệu có nhiều item luôn cùng xuất hiện (như tập $\{39, 40, 49\}$ trong retail).

3. Vector hóa SIMD cho phép AND bit

A2 hiện dùng các phép AND trên BitVector của Julia, đã được vector hóa ở mức trình biên dịch. Tuy nhiên, có thể tăng tốc thêm bằng cách dùng macro `@simd` cho các vòng lặp xử lý nhiều bucket song song, hoặc tận dụng AVX-512 trên CPU hỗ trợ. Kỳ vọng cải thiện 1.5 đến $2\times$ trên dữ liệu dày.

4. Đa luồng (multithreading)

Cây ppc-extension có thể được duyệt song song tại node gốc: các nhánh con khác nhau hoàn toàn độc lập về dữ liệu. Dùng `Threads.@threads` để phân chia các item mở rộng e cho nhiều luồng xử lý đồng thời. Kỳ vọng đạt tăng tốc gần tuyến tính theo số lõi vật lý trên các tập dữ liệu lớn (accidents).

4.9.4 Kết luận chương

Chương 4 đã trình bày đầy đủ sáu thực nghiệm bắt buộc với kết quả được lượng hóa bằng bảng số liệu và biểu đồ trực quan. Cài đặt Julia đạt độ chính xác 100% trên 76 trường hợp kiểm thử (bao gồm cả stress test) với tổng cộng hơn 937 nghìn tập đóng được đối chiếu. Về hiệu năng, mỗi phiên bản có vùng thể mạnh riêng, và khi kết hợp với chiến lược adaptive representation, có thể vượt SPMF từ 3 đến 200 lần tùy đặc tính dữ liệu. Bốn hướng tối ưu hóa được đề xuất hoàn toàn khả thi và phù hợp với cấu trúc thuật toán đã trình bày trong các chương trước.

CHƯƠNG 5

ỨNG DỤNG – PHÂN TÍCH GIỎ HÀNG

Để đánh giá khả năng ứng dụng thực tế của thuật toán LCM, đồ án tiến hành bài toán phân tích giỏ hàng trên tập dữ liệu bán lẻ thực tế `retail.txt`. Đây là bộ dữ liệu thường được sử dụng trong nghiên cứu khai thác dữ liệu, gồm các giao dịch mua hàng của khách trong siêu thị.

Mỗi giao dịch được biểu diễn dưới dạng một tập item:

$$t_i \subseteq I$$

trong đó:

- I là tập tất cả sản phẩm
- t_i là các sản phẩm được mua cùng nhau trong giao dịch thứ i

Mục tiêu của bài toán là:

1. Khai phá các tập phổ biến đóng bằng thuật toán LCM
2. Sinh các luật kết hợp từ các tập phổ biến tìm được
3. Xếp hạng luật theo chỉ số lift
4. Phân tích ý nghĩa kinh doanh của các luật mạnh

5.1 Cấu hình thực nghiệm

Đồ án sử dụng:

- Phiên bản thuật toán: A1 – TID-list
- Dữ liệu: `retail.txt`
- Số giao dịch:

$$|T| = 88162$$

- Ngưỡng hỗ trợ tối thiểu:

$$minsup = 1\%$$

- Ngưỡng confidence:

$$minconf = 0.5$$

Ngưỡng độ hỗ trợ tuyệt đối được tính:

$$s = 0.01 \times 88162 \approx 882.$$

Một itemset được xem là phổ biến nếu:

$$support(X) \geq 882.$$

Lệnh thực thi:

```
julia --project run_lcm.jl data/retail.txt "1%" rules a1 0.5
```

Việc lựa chọn ngưỡng $minsup = 1\%$ được thực hiện nhằm cân bằng giữa số lượng mẫu tìm được và thời gian xử lý. Nếu chọn ngưỡng độ hỗ trợ quá cao, nhiều mẫu mua hàng có giá trị sẽ bị loại bỏ. Ngược lại, nếu độ hỗ trợ quá thấp, số lượng itemset sinh ra tăng rất nhanh làm tăng chi phí khai phá và sinh luật. Vì dữ liệu có hơn 88 nghìn giao dịch, nên $minsup = 1\%$ tương đương 882 giao dịch. Đây không phải ngưỡng thấp về mặt số lượng tuyệt đối. Nó giúp cân bằng giữa số lượng mẫu thu được và chi phí tính toán, đồng thời đảm bảo các luật sinh ra đều dựa trên lượng dữ liệu đủ lớn để có ý nghĩa thống kê.

Ngưỡng $minconf = 0.5$ được lựa chọn để chỉ giữ lại các luật có độ tin cậy từ 50% trở lên, giúp loại bỏ các mối liên hệ yếu và tập trung vào các mẫu mua hàng có ý nghĩa thực tế.

5.2 Support, confidence và lift

Với luật $X \Rightarrow Y$ các độ đo được định nghĩa như sau:

Support

$$support(X \Rightarrow Y) = \frac{freq(X \cup Y)}{|T|}$$

Support cho biết tỷ lệ giao dịch chứa đồng thời cả X và Y .

Confidence

$$confidence(X \Rightarrow Y) = \frac{support(X \cup Y)}{support(X)}$$

Confidence biểu diễn xác suất xuất hiện của Y khi X đã xuất hiện.

Lift

$$lift(X \Rightarrow Y) = \frac{confidence(X \Rightarrow Y)}{support(Y)}$$

Lift đo mức độ phụ thuộc giữa hai tập sản phẩm:

- $lift > 1$: tương quan dương

- $lift = 1$: độc lập
- $lift < 1$: tương quan âm

Giá trị lift càng lớn thì mối liên hệ giữa hai tập sản phẩm càng mạnh.

5.3 Kết quả thực nghiệm

Kết quả chạy chương trình:

- Số tập phổ biến đóng tìm được: 160
- Số luật kết hợp sinh được: 124
- Thời gian khai phá: 9.12 giây

Kết quả cho thấy thuật toán LCM có thể xử lý hiệu quả tập dữ liệu lớn với hơn 88 nghìn giao dịch trong thời gian ngắn.

Bảng 5.1 trình bày 10 luật kết hợp có giá trị lift cao nhất.

Bảng 5.1: Top – 10 association rules theo lift

STT	Luật kết hợp	Support	Confidence	Lift
1	$\{40, 49, 111\} \Rightarrow \{39\}$	0.0117	0.9942	85.0164
2	$\{38\} \Rightarrow \{39\}$	0.0119	0.9739	82.0875
3	$\{37, 40, 49\} \Rightarrow \{39\}$	0.0123	0.9677	78.9982
4	$\{287\} \Rightarrow \{39\}$	0.0127	0.9434	74.5241
5	$\{40, 49, 171\} \Rightarrow \{39\}$	0.0135	0.9892	73.1028
6	$\{49, 102\} \Rightarrow \{40\}$	0.0107	0.7216	67.2479
7	$\{37, 39, 49\} \Rightarrow \{40\}$	0.0123	0.7941	64.8258
8	$\{39, 49, 111\} \Rightarrow \{40\}$	0.0117	0.7575	64.7774
9	$\{49, 111\} \Rightarrow \{39, 40\}$	0.0117	0.7471	63.8855
10	$\{49, 111\} \Rightarrow \{39\}$	0.0154	0.9862	63.8855

5.4 Phân tích các luật kết hợp nổi bật

Bảng 5.1 cho thấy các luật kết hợp có giá trị lift rất cao, dao động từ 63 đến 85. Trong khai phá luật kết hợp, lift là thước đo phản ánh mức độ phụ thuộc giữa hai tập sản phẩm. Giá trị lift càng lớn chứng tỏ các sản phẩm xuất hiện cùng nhau nhiều hơn đáng kể so với trường hợp độc lập. Vì vậy, các luật thu được không phải là những mối liên hệ ngẫu nhiên mà phản ánh những hành vi mua sắm có tính lặp lại trong thực tế.

Một đặc điểm nổi bật của kết quả là sản phẩm mang mã 39 xuất hiện trong phần lớn các luật có lift cao nhất. Chẳng hạn, luật:

$$\{40, 49, 111\} \Rightarrow \{39\}$$

có độ tin cậy đạt 99.42% và lift bằng 85.02. Điều này cho thấy gần như mọi giao dịch chứa đồng thời các sản phẩm 40, 49 và 111 đều xuất hiện thêm sản phẩm 39. So với xác suất xuất hiện thông thường của sản phẩm 39 trong toàn bộ cơ sở dữ liệu, xác suất này tăng hơn 85 lần khi nhóm sản phẩm bên trái xuất hiện.

Ngoài ra, các luật:

$$\{38\} \Rightarrow \{39\},$$

$$\{287\} \Rightarrow \{39\},$$

$$\{37, 40, 49\} \Rightarrow \{39\}$$

đều có confidence lớn hơn 94%. Điều này cho thấy sản phẩm 39 có mối liên hệ mạnh với nhiều nhóm sản phẩm khác nhau và đóng vai trò trung tâm trong mạng lưới mua sắm của khách hàng. Sự xuất hiện lặp lại của cùng một sản phẩm trong nhiều luật mạnh thường phản ánh một mặt hàng có tầm quan trọng đặc biệt trong hành vi tiêu dùng.

Bên cạnh đó, sản phẩm 40 cũng xuất hiện thường xuyên trong các luật có lift cao. Ví dụ:

$$\{49, 102\} \Rightarrow \{40\}$$

có confidence đạt 72.16% và lift bằng 67.25. Mặc dù confidence thấp hơn một số luật khác, giá trị lift vẫn rất lớn, cho thấy sự xuất hiện của sản phẩm 40 gắn liền với một số nhóm sản phẩm cụ thể thay vì xuất hiện ngẫu nhiên.

Một quan sát đáng chú ý khác là nhiều luật mạnh cùng chứa các sản phẩm 39, 40 và 49. Điều này cho thấy các sản phẩm này có thể thuộc cùng một nhóm nhu cầu hoặc thường xuyên được khách hàng lựa chọn trong cùng một mục đích mua sắm. Việc nhận diện các cụm sản phẩm như vậy có ý nghĩa quan trọng đối với các chiến lược bán hàng và marketing của doanh nghiệp.

5.5 Ý nghĩa kinh doanh

Kết quả khai phá luật kết hợp giúp doanh nghiệp hiểu rõ hơn hành vi mua sắm của khách hàng thông qua việc phát hiện các sản phẩm thường xuất hiện cùng nhau trong một giao dịch. Những tri thức này có thể được sử dụng để hỗ trợ nhiều hoạt động kinh doanh khác nhau.

5.5.1 Hỗ trợ gợi ý sản phẩm

Một trong những ứng dụng phổ biến nhất của luật kết hợp là xây dựng hệ thống gợi ý sản phẩm. Khi khách hàng lựa chọn một sản phẩm hoặc một nhóm sản phẩm, hệ thống có thể đề xuất các sản phẩm có khả năng được mua tiếp theo dựa trên các luật đã khai phá.

Ví dụ, luật:

$$\{38\} \Rightarrow \{39\}$$

có confidence đạt 97.39%. Điều này cho thấy phần lớn khách hàng mua sản phẩm 38 đều đồng thời mua sản phẩm 39. Vì vậy, khi khách hàng thêm sản phẩm 38 vào giỏ hàng, hệ thống có thể ưu tiên đề xuất sản phẩm 39 nhằm tăng khả năng phát sinh thêm giao dịch.

Tương tự, luật:

$$\{40, 49, 111\} \Rightarrow \{39\}$$

cho thấy sản phẩm 39 là lựa chọn gần như tất yếu khi khách hàng mua đồng thời ba sản phẩm còn lại. Những luật như vậy đặc biệt phù hợp cho các hệ thống recommendation trong thương mại điện tử.

5.5.2 Tăng doanh thu thông qua bán chéo sản phẩm

Bán chéo sản phẩm (cross-selling) là chiến lược khuyến khích khách hàng mua thêm các sản phẩm có liên quan nhằm gia tăng giá trị đơn hàng.

Kết quả thực nghiệm cho thấy nhiều nhóm sản phẩm có mối liên hệ rất mạnh. Chẳng hạn:

$$\{49, 102\} \Rightarrow \{40\}$$

có lift bằng 67.25. Điều này cho thấy khách hàng mua đồng thời sản phẩm 49 và 102 có xu hướng mua thêm sản phẩm 40 với xác suất cao hơn rất nhiều so với trung bình.

Từ kết quả này, doanh nghiệp có thể xây dựng các gói sản phẩm, hiển thị các đề xuất mua kèm hoặc áp dụng các chương trình ưu đãi khi khách hàng mua đồng thời các sản phẩm liên quan. Nhờ đó doanh thu trung bình trên mỗi giao dịch có thể được cải thiện đáng kể.

5.5.3 Thiết kế chương trình khuyến mãi

Các luật kết hợp có confidence cao phản ánh những nhóm sản phẩm thường xuyên được mua cùng nhau. Thay vì lựa chọn sản phẩm khuyến mãi dựa trên kinh nghiệm chủ quan, doanh nghiệp có thể dựa vào các mẫu mua sắm thực tế được phát hiện từ dữ liệu.

Ví dụ:

$$\{40, 49, 171\} \Rightarrow \{39\}$$

có confidence đạt 98.92%. Điều này cho thấy sản phẩm 39 gần như luôn xuất hiện cùng với nhóm sản phẩm còn lại. Doanh nghiệp có thể thiết kế các chương trình giảm giá theo nhóm, xây dựng các gói combo hoặc phát hành coupon dành cho khách hàng mua một phần của nhóm sản phẩm này.

Việc thiết kế khuyến mãi dựa trên dữ liệu thực tế thường mang lại hiệu quả cao hơn do phù hợp với hành vi mua sắm của khách hàng.

5.5.4 Tối ưu bố trí hàng hóa

Trong môi trường bán lẻ, các sản phẩm có mối liên hệ mua kèm mạnh nên được bố trí gần nhau nhằm tạo thuận lợi cho khách hàng và gia tăng khả năng mua thêm.

Các luật trong Top-10 cho thấy sản phẩm 39 xuất hiện cùng nhiều sản phẩm khác nhau với xác suất rất cao. Điều này cho thấy sản phẩm 39 có thể được xem như một sản phẩm trung tâm trong nhiều giao dịch mua sắm.

Doanh nghiệp có thể bố trí các sản phẩm liên quan ở các khu vực gần nhau hoặc hiển thị đồng thời trên giao diện bán hàng trực tuyến. Cách bố trí này giúp khách hàng dễ dàng nhận biết các sản phẩm liên quan và góp phần làm tăng doanh số bán hàng.

5.5.5 Hỗ trợ dự báo nhu cầu và quản lý tồn kho

Các luật kết hợp còn hỗ trợ doanh nghiệp trong việc dự báo nhu cầu sản phẩm. Khi một sản phẩm có quan hệ mạnh với nhiều sản phẩm khác, sự gia tăng nhu cầu của sản phẩm đó thường kéo theo sự gia tăng nhu cầu của các sản phẩm liên quan.

Ví dụ, sự xuất hiện lặp lại của sản phẩm 39 trong nhiều luật mạnh cho thấy đây là một sản phẩm quan trọng trong cơ sở dữ liệu giao dịch. Nếu nhu cầu đối với các sản phẩm liên quan tăng lên, doanh nghiệp có thể chủ động chuẩn bị tồn kho cho sản phẩm 39 nhằm tránh tình trạng thiếu hàng.

Khả năng dự báo nhu cầu dựa trên luật kết hợp đặc biệt hữu ích đối với các doanh nghiệp bán lẻ có số lượng mặt hàng lớn và nhu cầu biến động theo thời gian.

5.6 Nhận xét

Kết quả thực nghiệm cho thấy thuật toán LCM có khả năng khai phá hiệu quả các tập phổ biến đóng trên tập dữ liệu bán lẻ quy mô lớn gồm 88 162 giao dịch.

Với ngưỡng hỗ trợ tối thiểu 1%, thuật toán tìm được 160 closed frequent itemsets và sinh ra 124 luật kết hợp thỏa mãn điều kiện support và confidence. Toàn bộ quá trình khai phá hoàn thành trong khoảng 9 giây, cho thấy khả năng xử lý tốt đối với dữ liệu thực tế.

Các luật thu được đều có confidence rất cao (nhiều luật vượt 95%) và lift lớn hơn 60. Điều này chứng minh rằng các sản phẩm trong luật có mối quan hệ mua kèm

manh và mang giá trị thực tiễn đối với doanh nghiệp bán lẻ.

Kết quả cũng cho thấy việc khai phá trên closed frequent itemsets giúp giảm đáng kể số lượng mẫu cần lưu trữ so với khai phá toàn bộ frequent itemsets, trong khi vẫn bảo toàn thông tin cần thiết để sinh luật kết hợp.

Nhìn chung, thuật toán LCM không chỉ đạt hiệu quả về mặt tính toán mà còn tạo ra các tri thức có giá trị cho các bài toán recommendation, cross-selling, thiết kế khuyến mãi, tối ưu bố trí hàng hóa và phân tích hành vi khách hàng trong môi trường bán lẻ thực tế.

TÀI LIỆU THAM KHẢO

- [1] Rakesh Agrawal and Ramakrishnan Srikant. Fast algorithms for mining association rules in large databases. In *Proceedings of the 20th International Conference on Very Large Data Bases (VLDB 1994)*, pages 487–499. Morgan Kaufmann, 1994.
- [2] Jiawei Han, Jian Pei, and Yiwen Yin. Mining frequent patterns without candidate generation. In *Proceedings of the ACM SIGMOD International Conference on Management of Data (SIGMOD 2000)*, pages 1–12. ACM, 2000.
- [3] Takeaki Uno, Masashi Kiyomi, and Hiroki Arimura. LCM ver. 2: Efficient mining algorithms for frequent/closed/maximal itemsets. In *Proceedings of the IEEE ICDM Workshop on Frequent Itemset Mining Implementations (FIMI 2004)*, volume 126 of *CEUR Workshop Proceedings*, 2004.
- [4] Mohammed J. Zaki and Ching-Jui Hsiao. CHARM: An efficient algorithm for closed itemset mining. In *Proceedings of the 2nd SIAM International Conference on Data Mining (SDM 2002)*, pages 457–473. SIAM, 2002.